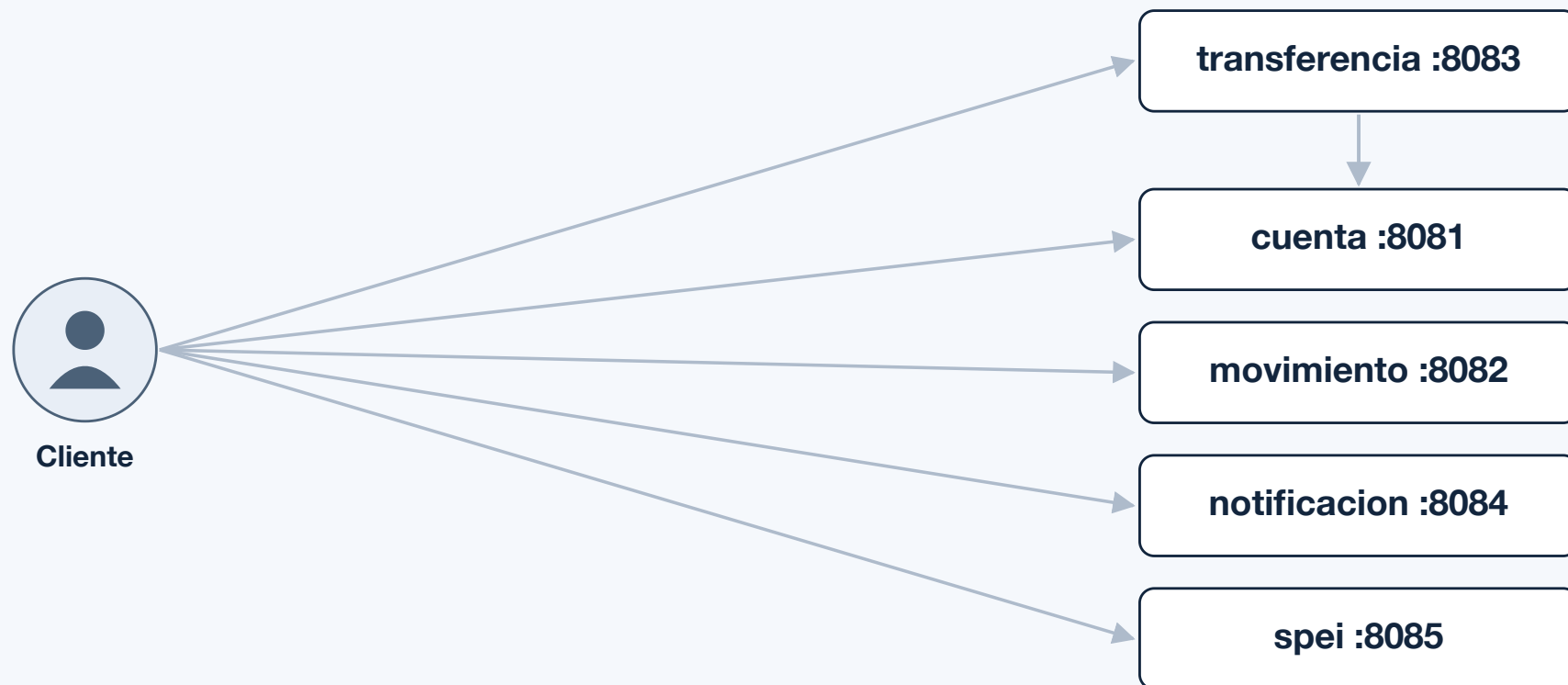
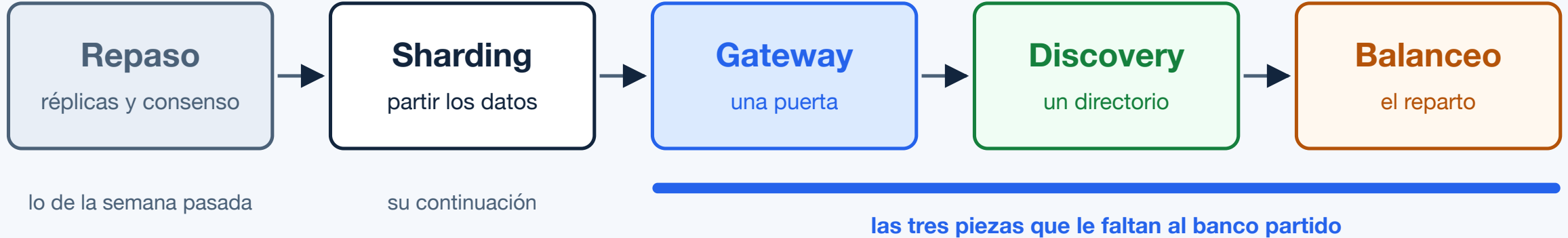


Una puerta, un directorio y el reparto



Hoy el banco partido vuelve a hablar

Cinco paradas y un laboratorio



Después del descanso: ustedes arman las tres piezas en el laboratorio

Empezamos por donde nos quedamos: las copias

Replicar: la misma información en varias máquinas

Para qué se replica

Estar cerca del usuario: responde más rápido

Seguir funcionando si una máquina se cae

Atender más lecturas de las que aguanta una sola

Lo que no es

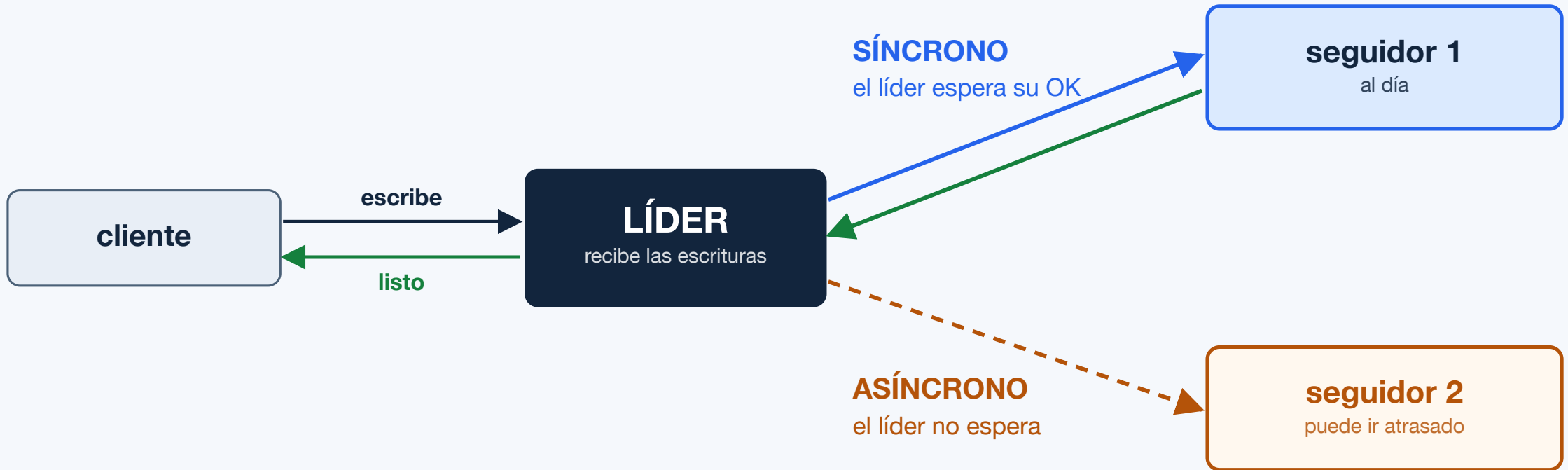
Una réplica no es un respaldo

Si borras algo por error, el borrado también se copia

El respaldo guarda una foto de cómo estaba antes

Copiar una vez es fácil. Lo difícil es copiar cada cambio, y ahí empieza todo

Un líder recibe las escrituras; los seguidores las copian



Síncrono: espera. Asíncrono: no espera. Si todos fueran síncronos, uno caído detiene todo

Leer de un seguidor atrasado da tres sorpresas

Lo que le pasa al usuario	La garantía que lo evita	Una forma de lograrla
Deposita y, al recargar, no ve su depósito	Leer tus propias escrituras	Lo tuyo se lee del líder
Ve un movimiento y, al recargar, ya no está	Lecturas monótonas	Lees siempre de la misma copia
Ve la respuesta antes que la pregunta	Prefijo consistente	Lo relacionado se escribe en el mismo lugar, en orden

Consistencia eventual: los seguidores se ponen al día, pero nadie dice cuándo

Tres formas de replicar, según quién acepta las escrituras

Familia	Quién acepta escrituras	Lo que pagas	Ejemplo
Un líder	Uno solo; los demás copian	Si el líder cae, hay que relevarlo	PostgreSQL, MySQL, Kafka
Varios líderes	Uno por región o por dispositivo	Dos escriben lo mismo a la vez: conflictos	Google Docs, el calendario del celular
Sin líder	Cualquier copia; se escribe y se lee de varias	Puedes leer algo viejo; también hay conflictos	Cassandra, Dynamo

Un líder evita los conflictos. Las otras dos aguantan mejor que se corte la red

Si se cae el líder, hay que nombrar otro, y ahí está el peligro

Los tres pasos del relevo

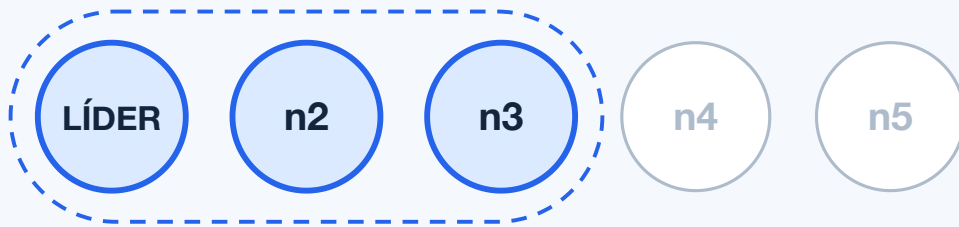
- 1 · Darse cuenta: el líder dejó de mandar heartbeat y venció el plazo
- 2 · Elegir al seguidor más al día
- 3 · Avisar a todos quién es el nuevo líder

Lo que sale mal

Lo que el líder viejo no alcanzó a copiar se pierde
Dos nodos creen ser el líder y los dos aceptan escrituras
Plazo corto: relevos de más. Plazo largo: mucho tiempo sin líder

Nombrar un líder nuevo sin que haya dos a la vez: ese problema se llama consenso

Consenso: la mayoría decide dos veces



mayoría: 3 de 5

3 nodos toleran 1 caído

5 nodos toleran 2 caídos

Votación 1 · elegir líder

Gana quien junta la mayoría de los votos
Un voto por nodo en cada término

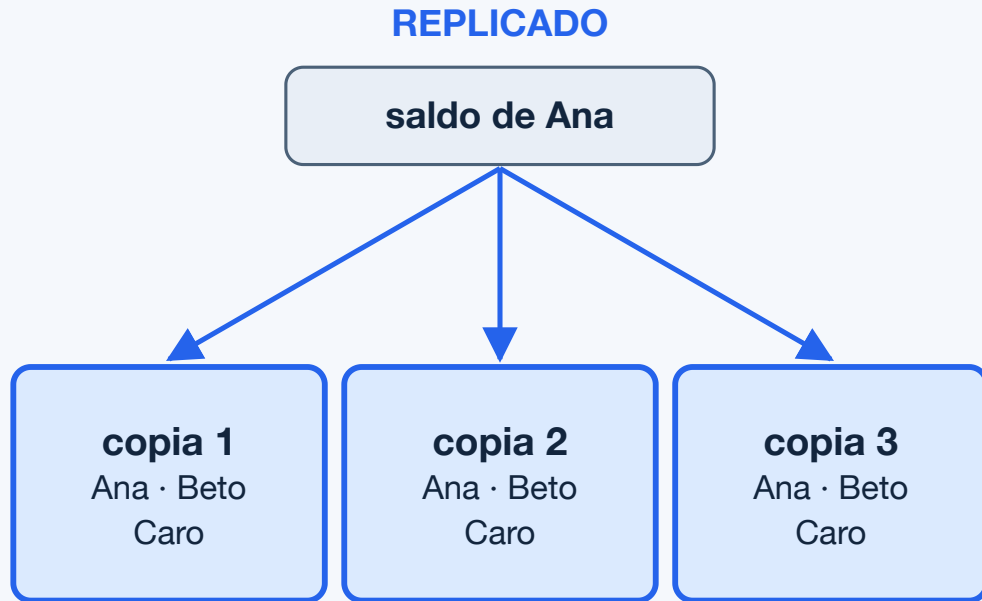
Votación 2 · dar por buena una escritura

Vale cuando la mayoría ya la guardó
Solo entonces se le confirma al cliente

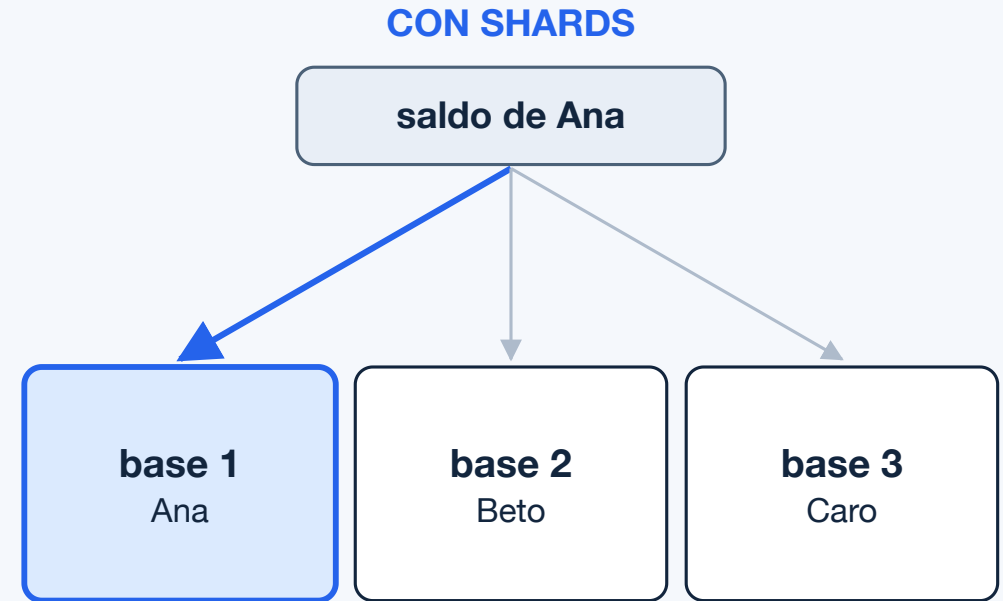
Así funciona Raft. Lo usa etcd, donde Kubernetes guarda su estado.

Nunca dos líderes en el mismo término, y lo confirmado no se pierde. Si no hay mayoría, se detiene

Replicar copia lo mismo; partir reparte cosas distintas



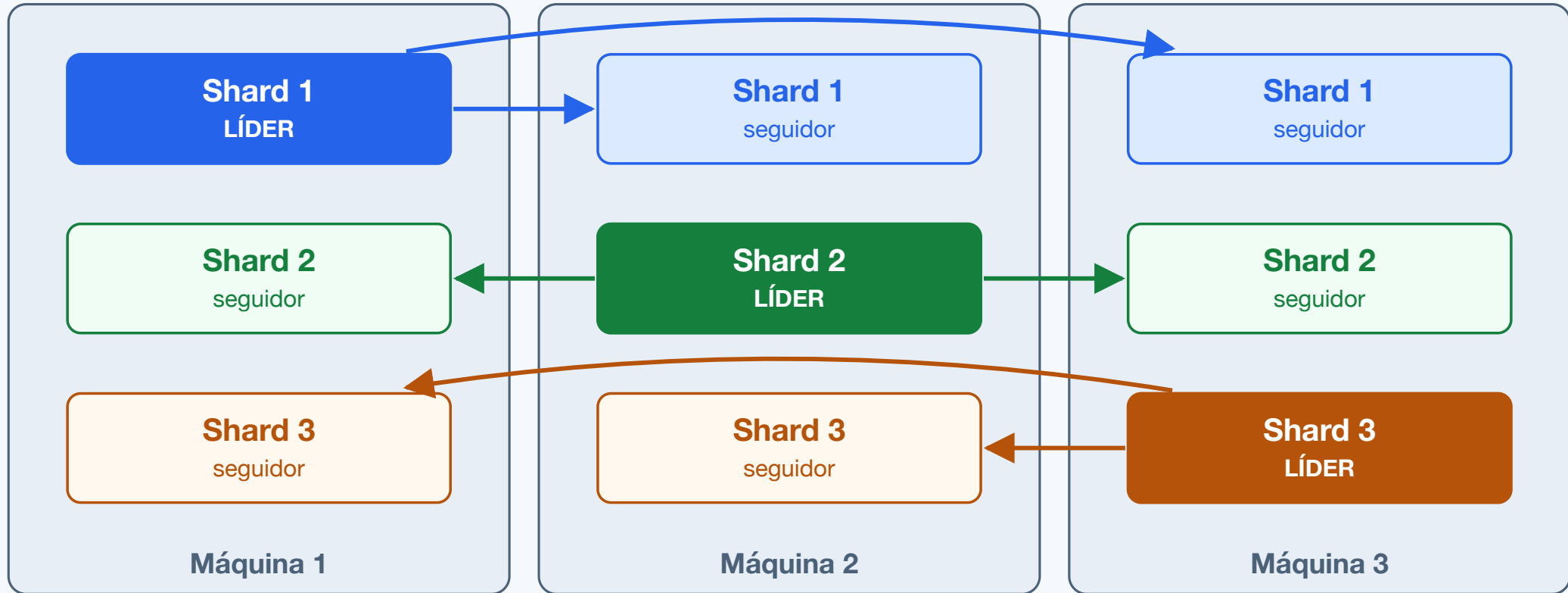
Cualquier copia puede contestar



Solo la base 1 tiene a Ana

A la izquierda cualquiera sirve. A la derecha, solo una tiene la respuesta

Cada shard tiene su líder y sus copias



Cada cuenta vive en un solo shard. Lo de la semana pasada ocurre dentro de cada color

Se parte cuando los datos ya no caben en una máquina

Las cuentas de Mexi Banco
tres partes de 100 GB: 300 GB

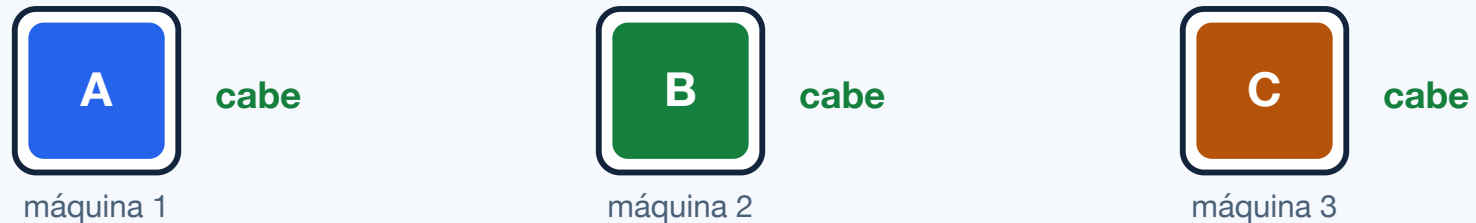


Cada máquina
tiene disco para 100 GB

Replicar
cada máquina guarda todo



Partir
cada máquina guarda una parte



También se parte cuando llegan más escrituras de las que aguanta una máquina. Si el problema son solo las lecturas, bastan réplicas.

Replicar no da más espacio: cada copia guarda todo. Partir sí

La llave de partición decide a qué parte va cada cosa



Si buscas por otro dato, por ejemplo todos los movimientos del 3 de octubre, hay que preguntarles a las tres bases.

Misma llave, misma base: todo lo de Ana queda junto y se encuentra de un solo viaje

Una buena llave junta lo que es igual y separa lo que es distinto

Cada movimiento trae **cuenta**, **fecha** y **número de movimiento**. ¿Con qué dato los reparto entre tres bases?



Llave compuesta: el primer dato decide la base; el segundo ordena dentro de ella

Dónde se usa: cuatro sistemas, cuatro llaves

Sistema	Qué se reparte	La llave	Para qué
Mexi Banco	Las cuentas y sus movimientos	La CLABE	Los datos ya no caben en una máquina
Un Notion para muchas empresas	Los datos de cada empresa, en su propia parte	La empresa	Lo que hace una empresa no afecta a las demás (<i>multitenancy</i>)
Un videojuego de mundo abierto	Los jugadores, por zona del mapa	La ubicación	Los que están cerca juegan en el mismo servidor
Un Instagram	Las publicaciones de cada usuario	El usuario	Todo lo de un usuario queda junto

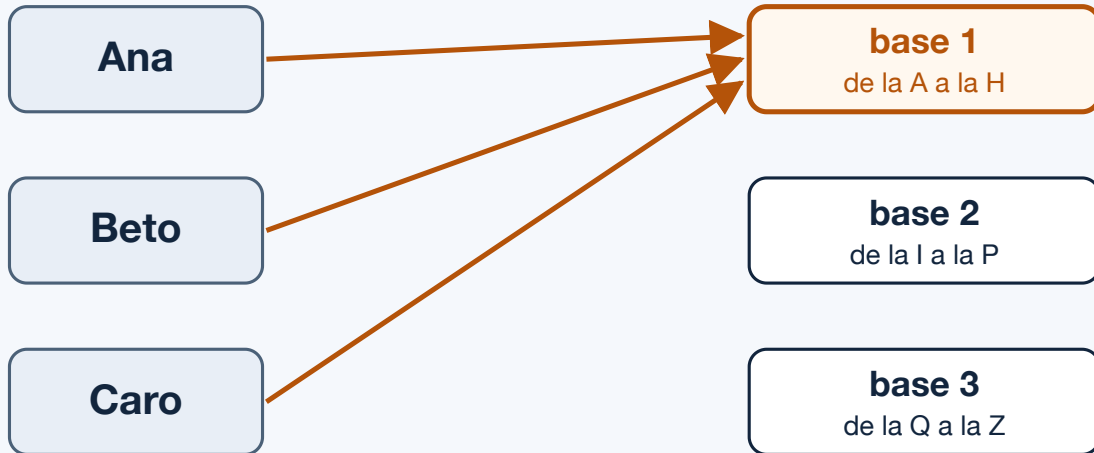
La misma idea aparece en las particiones de Kafka, en las cachés grandes y en los buscadores.

¿Por qué dato partirían el sistema de su proyecto?

Ya tengo la llave. Falta decidir cómo se convierte en una base

POR RANGO

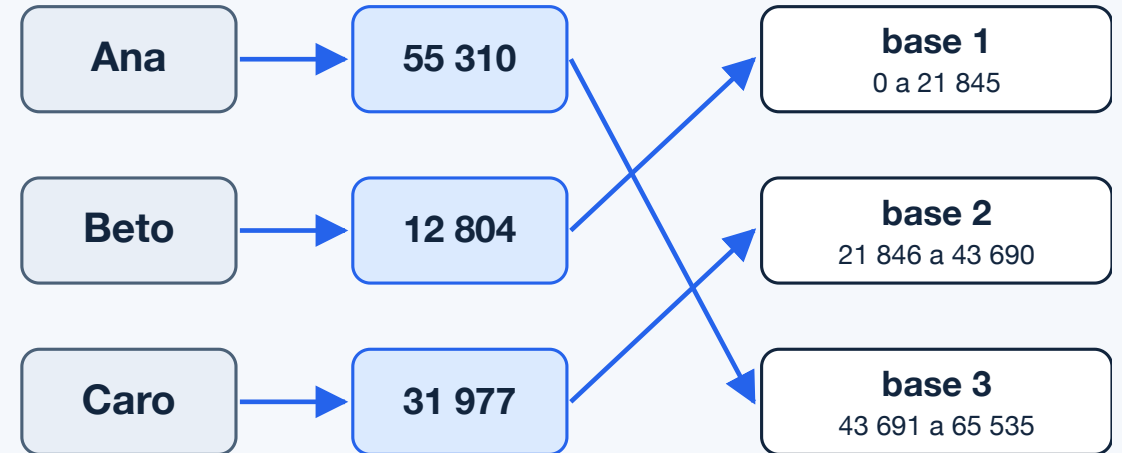
la llave se compara con los límites de cada base



Ana, Beto y Caro son vecinos: caen juntos

POR HASH

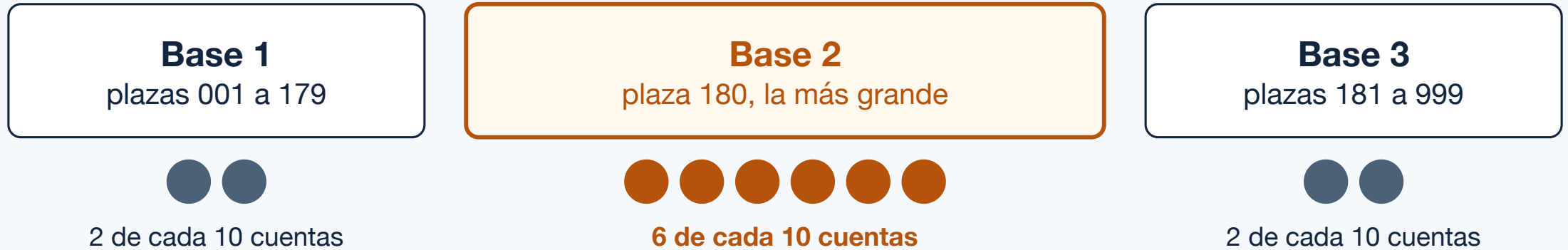
la llave pasa por una función que la convierte en un número



Los vecinos quedan repartidos

Por rango, lo vecino queda junto. Por hash, se revuelve. Cada una gana algo y paga algo

Por rango: lo vecino queda junto, y se amontona



Lo que gana

Buscar un rango toca una sola base: todas las cuentas de una plaza, los movimientos de un mes

Los límites se pueden mover cuando una base crece

Lo que paga

Si muchas llaves vecinas reciben carga, se amontonan en una base: la plaza grande

Con llaves que siempre crecen, como la fecha, todo lo nuevo cae en la última base

Conviene cuando vas a buscar por rangos. Cuidado con las llaves que solo crecen

Por hash: se reparte parejo, y se pierde el orden



Lo que gana

Reparto parejo, aunque las llaves se parezcan mucho
Las llaves que solo crecen ya no se amontonan

Lo que paga

Se pierde el orden: buscar un rango obliga a preguntarles a todas las bases

Misma llave, mismo número, siempre. Conviene cuando no necesitas buscar por rangos

El hash reparte parejo las cuentas, no el trabajo

Doce cuentas repartidas por hash entre tres bases. A la derecha de cada cuenta, sus operaciones por minuto en el Buen Fin.

Base 1	
Tienda grande	1 000
Ana	2
Eva	1
Luis	3

4 cuentas

1 006 operaciones por minuto

Base 2	
Beto	2
Caro	1
Iván	3
Sara	2

4 cuentas

8 operaciones por minuto

Base 3	
Dani	1
Fer	2
Gabi	2
Hugo	1

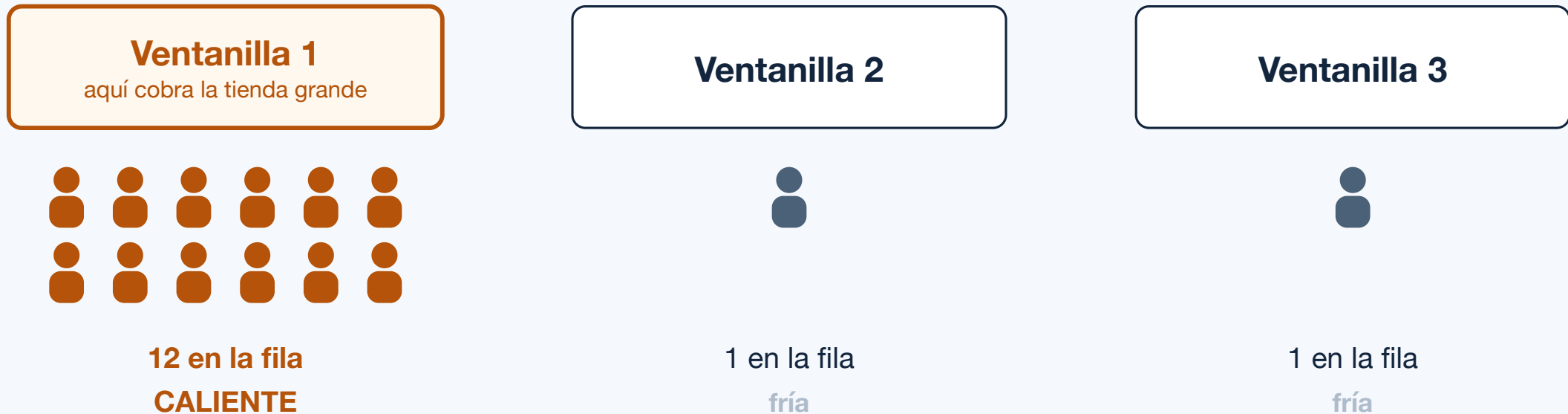
4 cuentas

6 operaciones por minuto

Cuatro cuentas en cada base: parejo. Pero una sola cuenta trae casi todo el trabajo

Tres ventanillas abiertas y toda la fila en una

El banco en pleno Buen Fin: tres ventanillas, pero casi todos los que llegan van a la misma.



Shard caliente: recibe mucha más carga que los demás. **Shard frío:** casi no recibe. **Llave caliente:** una sola llave que trae casi toda la carga.

Tener tres máquinas no sirve de nada si el trabajo le cae a una sola

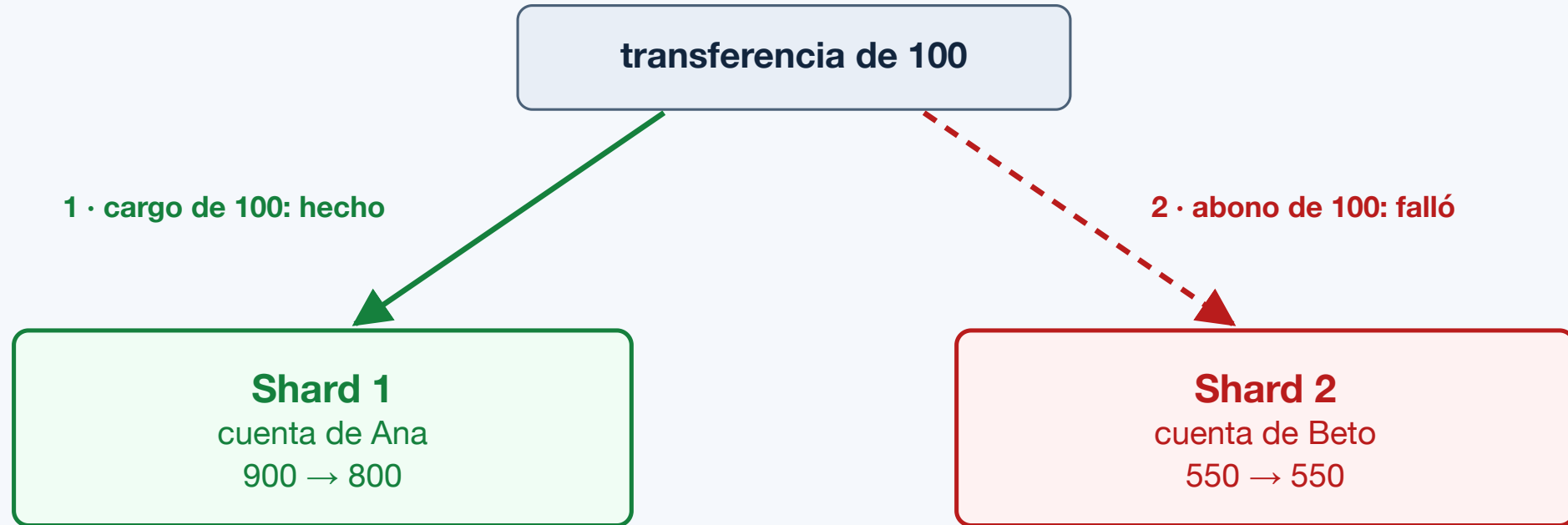
A la cuenta caliente se le da su propia máquina



Otras dos salidas: si lo caliente son lecturas, se agregan copias de ese shard; o se parte la cuenta en subcuentas, y para saber el saldo hay que sumarlas.

Las demás cuentas dejan de sufrir. Antes del Buen Fin, esto se hace a mano y con calma

Una transferencia entre dos shards ya no es una sola transacción

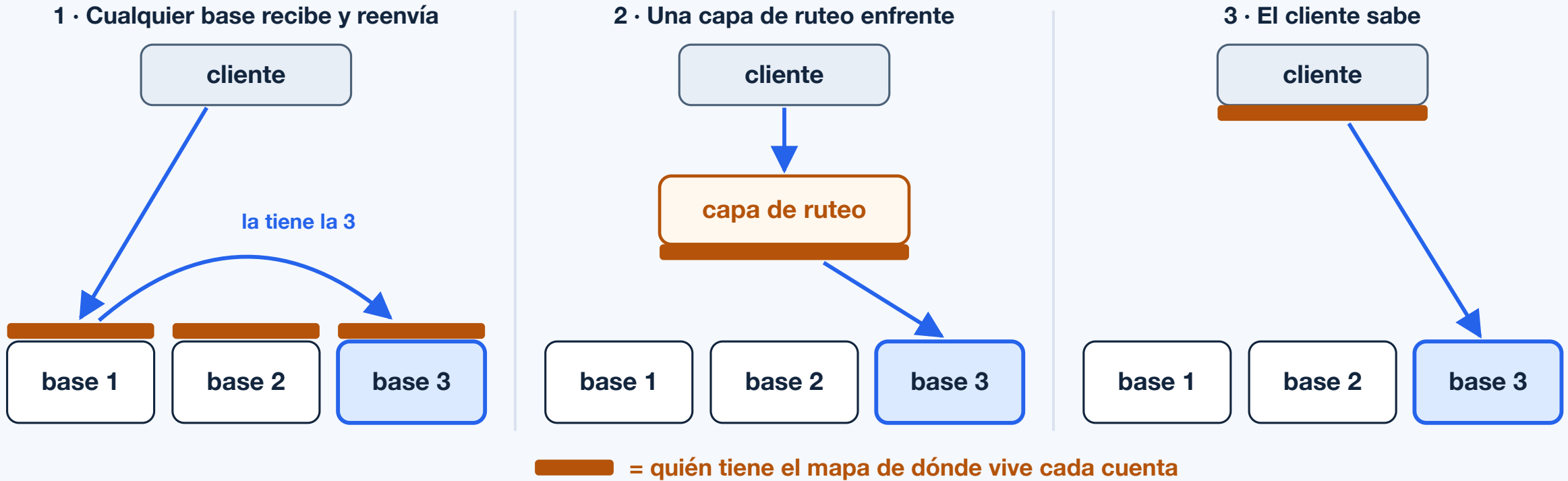


En una sola máquina, las dos escrituras ocurren juntas o ninguna. Entre dos shards, eso ya no viene gratis.

Los otros dos costos: buscar sin la llave obliga a preguntar a todos los shards, y la llave es difícil de cambiar después.

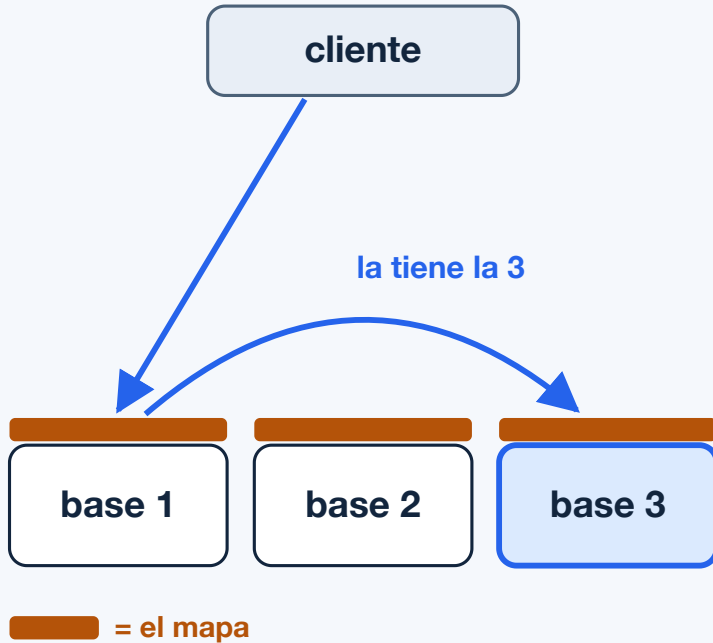
Es la misma clase de falla que van a ver hoy en la demo del banco partido

Alguien tiene que saber dónde está cada cosa



Tres formas. Vamos una por una: quién corre el código y cómo se ve

Cualquier base recibe; si la cuenta no es suya, reenvía

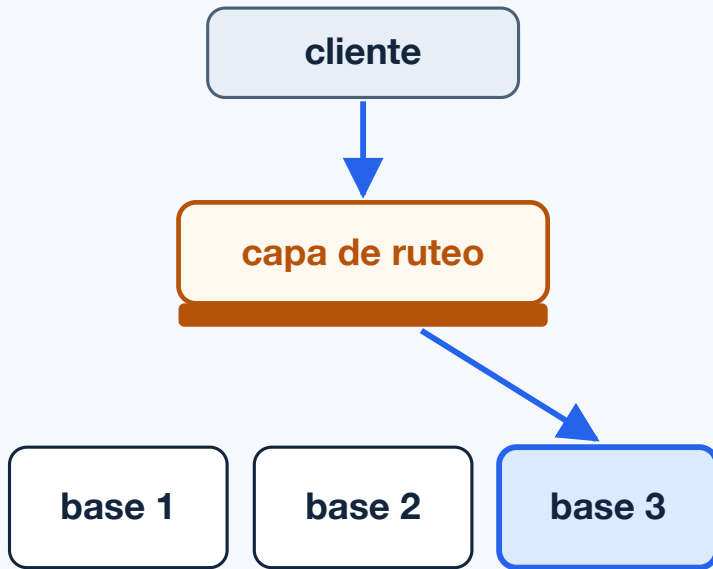


```
# corre en CADA base
def atender(peticion):
    shard = shard_de(peticion.llave)
    dueña = mapa[shard]
    if dueña == Y0:
        return leer(peticion.llave)
    return reenviar(dueña, peticion)
```

Quién lo hace así: Cassandra. Redis Cluster hace una variante: no reenvía, contesta "esa cuenta está en la base 3" y el cliente va.

Sin piezas extra. Pero todas las bases cargan con el mapa, y a veces hay un salto de más

Una capa enfrente que no guarda datos: solo sabe a quién mandar



 = el mapa

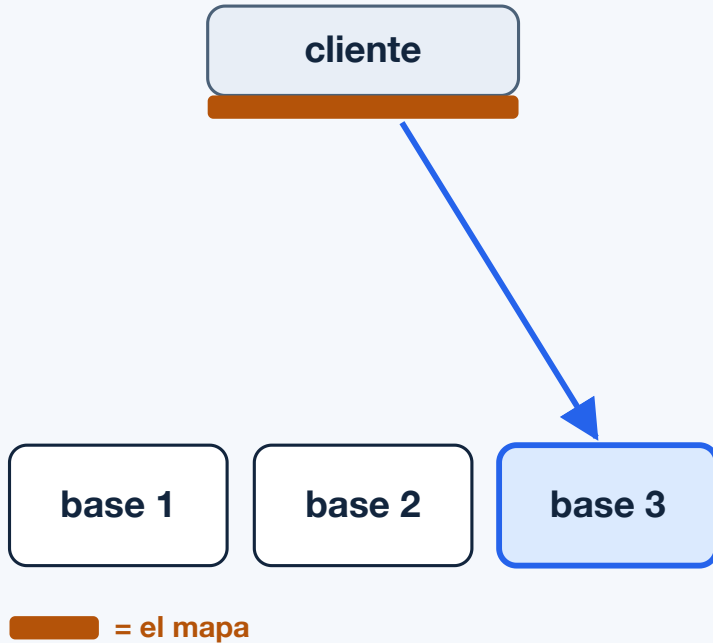
```
# corre en la capa de ruteo
mapa = coordinador.leer_mapa()
coordinador.avisame_si_cambia(actualizar)

def atender(peticion):
    shard = shard_de(peticion.llave)
    return reenviar(mapa[shard], peticion)
```

Quién lo hace así: MongoDB (el proceso mongos), Vitess sobre MySQL, twemproxy frente a memcached.

El cliente no sabe nada. Pero siempre hay un salto más, y esa capa no puede caerse

El cliente trae el mapa y va directo a la base correcta



```
# corre dentro del cliente (una biblioteca)
mapa = pedir_mapa()

def leer(llave):
    base = mapa[shard_de(llave)]
    try:
        return base.leer(llave)
    except YaNoEsMia:          # mi mapa estaba viejo
        mapa = pedir_mapa()
        return leer(llave)
```

Quién lo hace así: El productor de Kafka, que le escribe directo al líder de la partición; los clientes de Cassandra y de Redis Cluster.

Sin saltos de más. Pero el mapa que trae el cliente puede estar viejo

El mapa también es un dato, y alguien tiene que guardarlo

Sistema	Quién rutea	Dónde vive el mapa	Cómo se cuida
MongoDB	Una capa enfrente (mongos)	Servidores de configuración	Consenso
Kafka	El cliente	Los controladores del clúster	Consenso (Raft)
HBase	El cliente	ZooKeeper y una tabla índice	Consenso
Cassandra	Cualquier nodo	En todos los nodos	Chismes entre nodos

Con consenso (S06) no puede haber dos mapas distintos. Con chismes (*gossip*), cada nodo le cuenta a otros lo que sabe: el mapa llega a todos, pero tarde.

Si dos partes del sistema tienen mapas distintos, la cuenta de Ana aparece en dos lugares

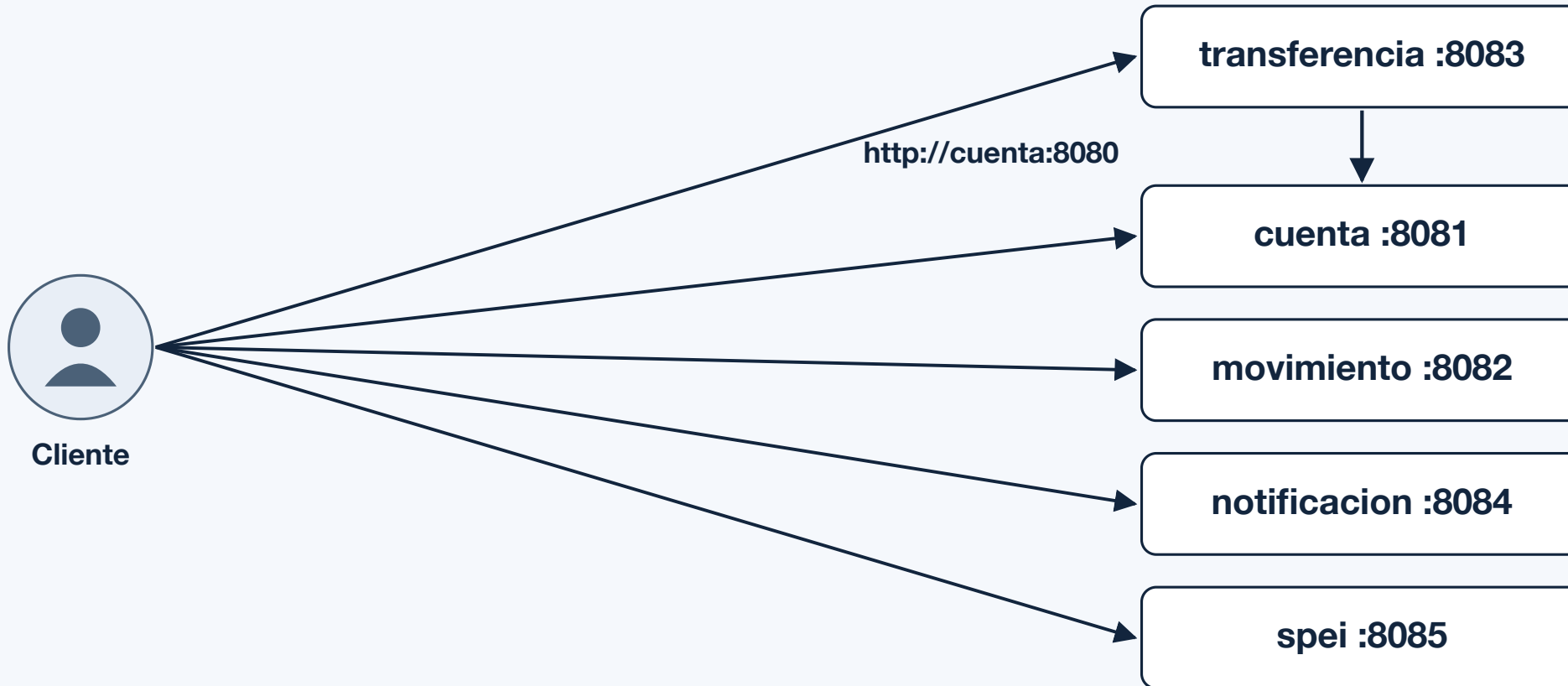
Las mismas tres formas, ahora para los servicios de Mexi Banco

En una base partida	En los servicios de hoy	La pieza en la etapa 03
Una capa de ruteo enfrente	Una sola puerta para el cliente	Gateway
El mapa, y quién lo cuida	Un directorio donde cada servicio se anota	Discovery
El cliente trae el mapa y elige	El que llama elige a qué copia hablarle	Balanceo del lado del cliente

La diferencia: una copia de un servicio sin estado atiende cualquier petición, así que no hay que mirar la llave. Con datos, solo sirve la base que tiene esa cuenta.

Primero veamos qué se rompe cuando partimos el banco

Así quedó Mexi Banco después de partirlo



Este dibujo no se mueve en toda la clase: hoy le agregamos tres piezas

Cinco servicios, cinco puertos publicados

```
$ docker compose ps
mexi-banco-v06a-cuenta-1      0.0.0.0:8081->8080/tcp
mexi-banco-v06a-movimiento-1  0.0.0.0:8082->8080/tcp
mexi-banco-v06a-transferencia-1 0.0.0.0:8083->8080/tcp
mexi-banco-v06a-notificacion-1 0.0.0.0:8084->8080/tcp
mexi-banco-v06a-spei-1        0.0.0.0:8085->8080/tcp

$ curl localhost:8081/cuentas/... # el saldo, en cuenta
$ curl localhost:8083/transferencias # transferir, en transferencia
```

Problema 1: el cliente tiene que sabérselos todos

Transferencia trae escrito a quién le habla

```
# docker-compose.yml de la etapa 02
transferencia:
  ports:
    - "8083:8080"
  environment:
    CUENTA_URL: http://cuenta:8080
    MOVIMIENTO_URL: http://movimiento:8080
    NOTIFICACION_URL: http://notificacion:8080
```

Problema 2: ese nombre no dice cuántas copias hay ni cuáles siguen vivas

PREGUNTA

Se cae cuenta a media transferencia de 100 pesos. ¿Qué pasa con el dinero?



Cliente

cargo hecho... ¿y el abono?

transferencia :8083



cuenta :8081

movimiento :8082

notificacion :8084

spei :8085

Escríbanlo en el chat antes de que corra la demo

A Ana le cobraron 100 que Beto nunca recibió

```
$ ./demo-v06a.sh --roto
```

```
17. Transferir 100 de Ana a Beto y, a media transferencia, apagar cuenta  
[HTTP 503]
```

```
antes: Ana + Beto = 1 450
```

```
ahora: Ana + Beto = 1 350
```

```
a Ana se le cobraron 100 que Beto nunca recibió
```

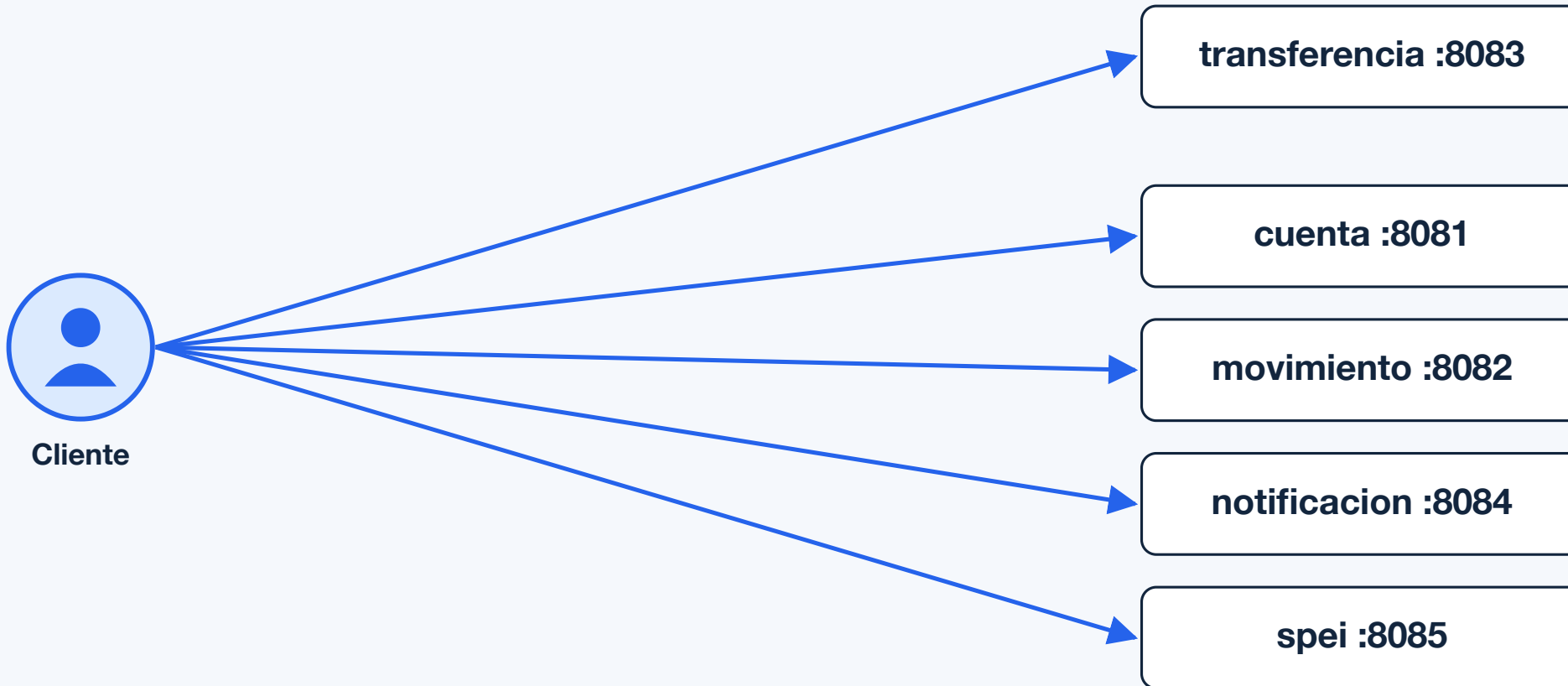
Hoy no lo arreglamos: timeouts en la S08 y sagas en la S12

Tres problemas, tres piezas

El problema en el banco partido	La pieza	Lo nuevo que puede fallar
El cliente se sabe cinco puertos	Gateway	Una puerta por donde pasa todo
El nombre cuenta no dice cuántas copias hay ni cuáles viven	Discovery	Un directorio que se entera tarde
Tres copias de cuenta: ¿a cuál llamo?	Balanceo	Repartir con una lista vieja

Cada pieza resuelve uno y trae un riesgo nuevo

El cliente se sabe cinco puertos



¿Qué pasa si mañana cuenta se muda al 9090? ¿Y si la app ya está instalada en mil celulares?

PREGUNTA

Partimos cuenta en dos servicios. ¿Quién tiene que cambiar?

A Solo el backend

B La app de cada cliente

C Nadie

Voten A, B o C en el chat

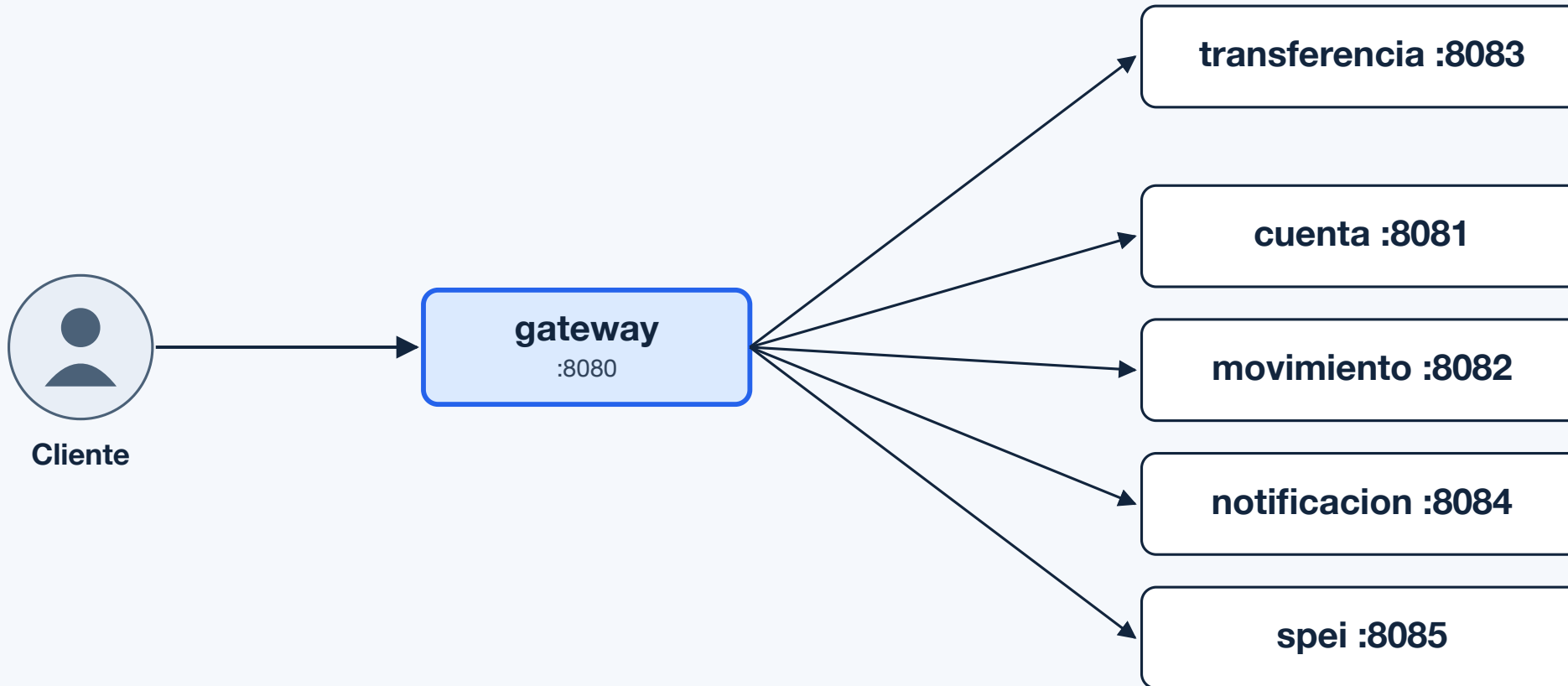
Alguien recibe la petición y la reenvía por ti



El cliente solo ve al intermediario. No sabe cuántos servidores hay ni dónde están.
Como el conmutador de una oficina: marcas un número y te pasan a la extensión.

Eso es un proxy inverso. El gateway y el balanceador son esto, con más o menos tareas

Una sola puerta para el cliente



Es la capa de ruteo de hace un rato: allá miraba la llave; aquí mira la ruta

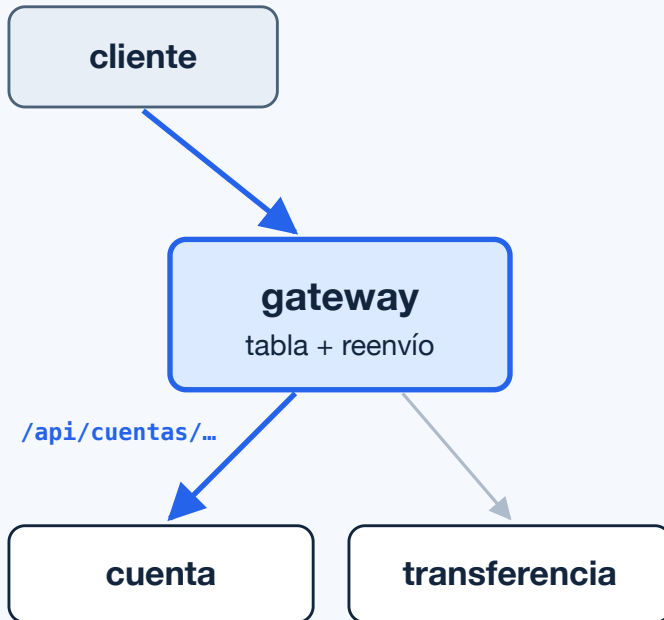
El gateway busca la ruta en su tabla

Si llega...	va a...
/api/cuentas/{clabe}	cuenta
GET /api/movimientos	movimiento
/api/transferencias	transferencia
GET /api/notificaciones	notificacion
POST /api/spei	spei

Quita `/api` y reenvía: cuenta recibe `/cuentas/{clabe}`. Es el `application.yml` del gateway en la etapa 03, y es lo que escriben en el lab (TODO H2).

La tabla es configuración, no código: se cambia sin tocar al cliente

Si lo escribiéramos nosotros: una tabla y un reenvío



```
TABLA = {  
    "/api/cuentas":      "http://cuenta:8080",  
    "/api/transferencias": "http://transferencia:8080",  
    "/api/movimientos":  "http://movimiento:8080",  
}  
  
def atender(peticion):  
    for prefijo, destino in TABLA.items():  
        if peticion.ruta.startswith(prefijo):  
            ruta = peticion.ruta.removeprefix("/api")  
            return reenviar(destino + ruta, peticion)  
    return 404          # sin ruta: no pasa
```

Con esto ya enruta. Lo que falta es lo difícil: tiempos de espera, reintentos, límites, HTTPS, registro de cada petición y correr varias copias.

Enrutar son doce líneas. Todo lo demás es la razón por la que no se escribe: se usa uno ya hecho

La misma fila de la tabla, en tres tecnologías

Spring Cloud Gateway

el del lab · application.yml

```
routes:
  - id: cuenta
    uri: lb://cuenta
    predicates:
      - Path=/api/cuentas/{clabe}
    filters:
      - StripPrefix=1
```

Pregunta al directorio dónde está cuenta

nginx

un archivo de configuración

```
location /api/cuentas/ {
    proxy_pass
        http://cuenta:8080/cuentas/;
}
```

La dirección va escrita, o la resuelve el DNS

Kubernetes

un Ingress

```
paths:
  - path: /api/cuentas
    pathType: Prefix
    backend:
      service:
        name: cuenta
        port:
          number: 8080
```

La plataforma ya sabe dónde está cuenta

Otros: Kong, Traefik y Envoy. El gateway no depende del lenguaje de los servicios: si su proyecto es en Python o en Node, cualquiera de estos les sirve.

En las tres es lo mismo: si llega esta ruta, va a este servicio. Se configura, no se programa

En la nube, el gateway se renta ya hecho

AWS

el gateway

API Gateway

el balanceador que también enruta por ruta

Application Load Balancer

Google Cloud

el gateway

API Gateway · Apigee

el balanceador que también enruta por ruta

Application Load Balancer

Azure

el gateway

API Management

el balanceador que también enruta por ruta

Application Gateway

Lo que cambia

No corres las copias: el proveedor las escala

El HTTPS, las credenciales y los límites se configuran, no se instalan

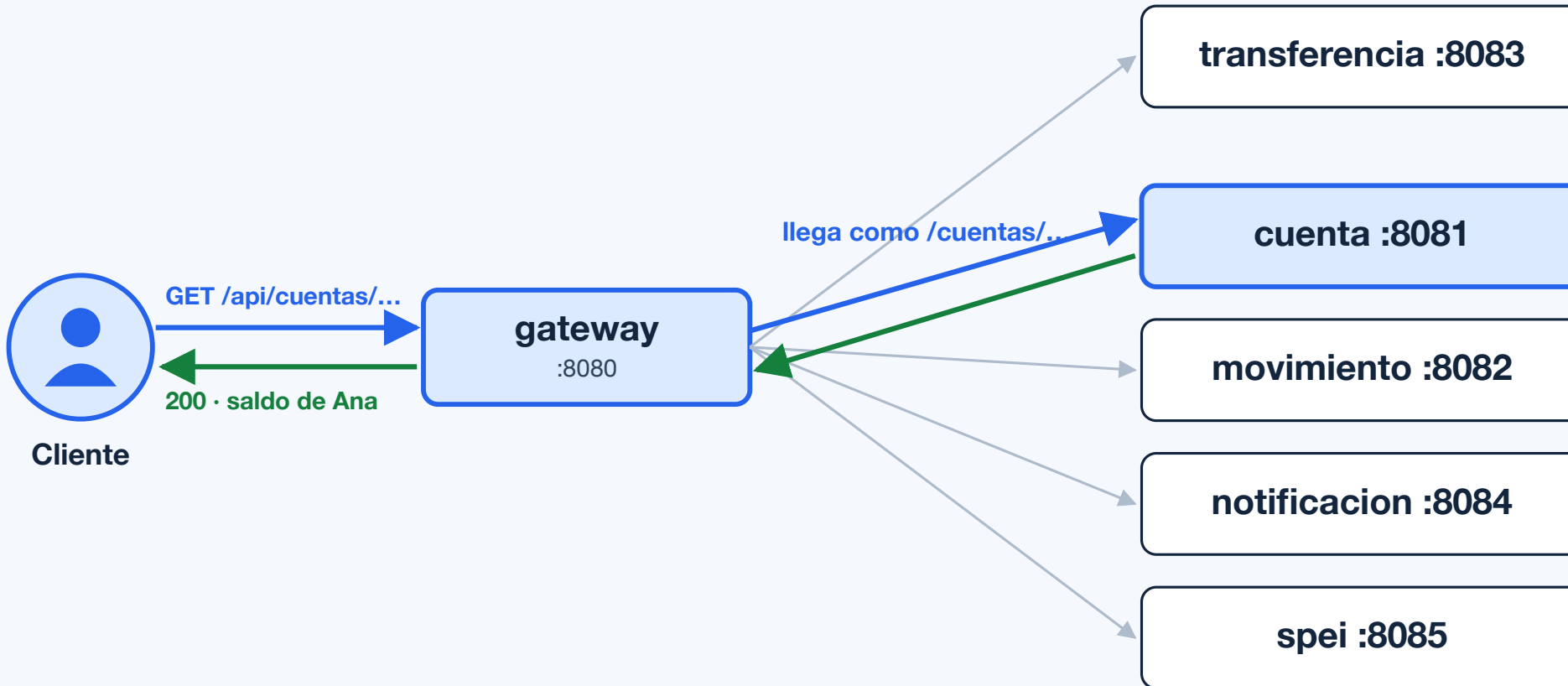
Lo que pagas

Se cobra por uso: cada petición cuenta

Quedas atado a ese proveedor

Misma idea y misma tabla de rutas. Cambia quién lo opera: tú o el proveedor

La petición entra y la respuesta sale por la misma puerta



El cliente no sabe el puerto de cuenta, ni dónde está, ni cuántas hay

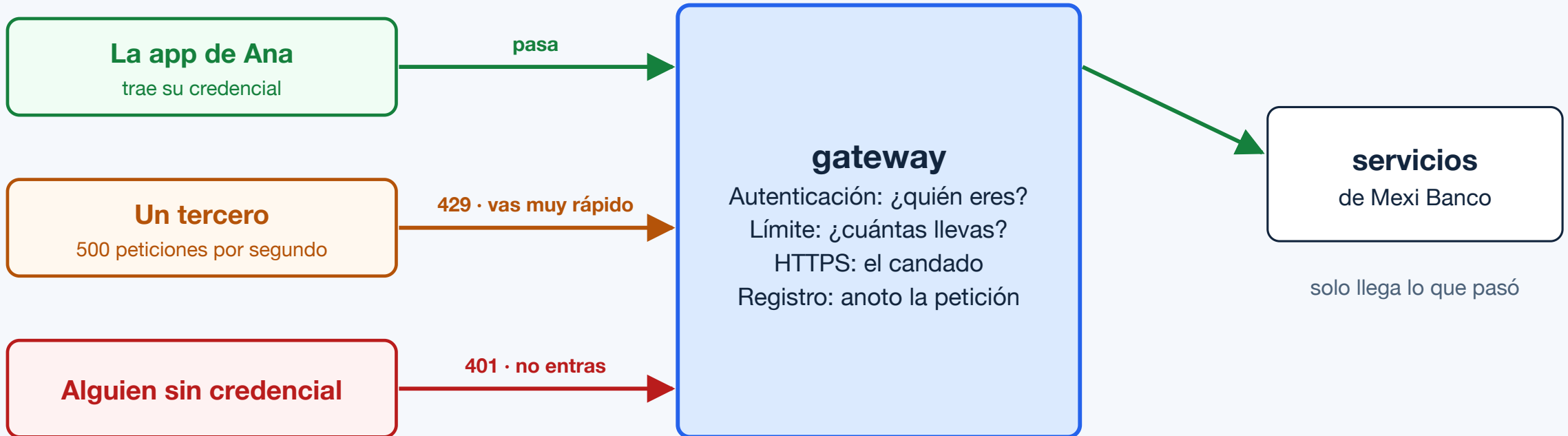
Con la puerta enfrente, el monolito se parte sin avisarle al cliente



Cada servicio que sale del monolito es una fila más en la tabla, hasta que el monolito queda vacío (patrón *strangler fig*). Así van a partir su proyecto.

Respuesta: A. Cambia el backend y una fila de la tabla; la app del cliente, no

En la puerta se revisa lo que todos los servicios necesitan

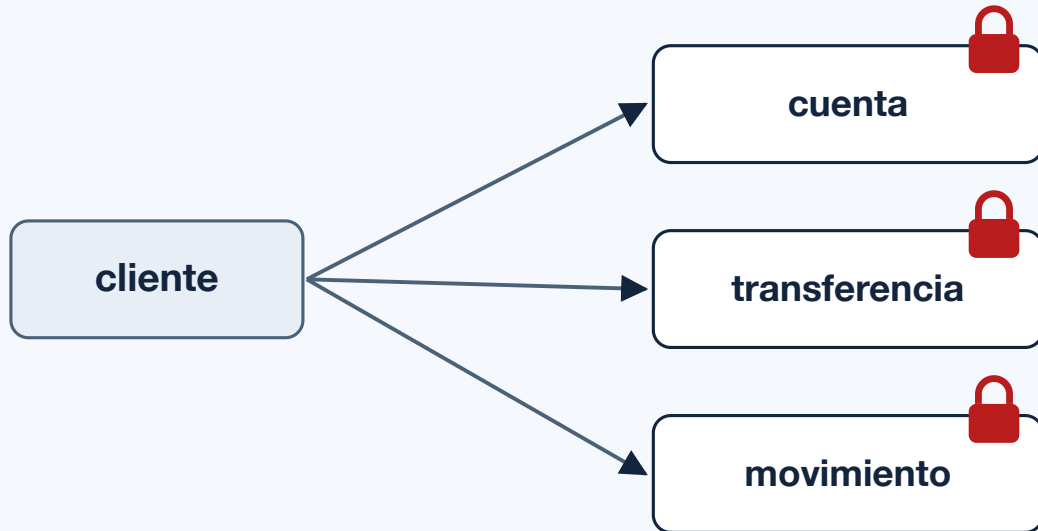


Es lo que un gateway puede hacer; el de la etapa 03 solo enruta. También le pone a cada petición un identificador para seguirla entre servicios: lo usamos en la S08.

Cuatro tareas que antes repetía cada servicio. Veamos las dos primeras

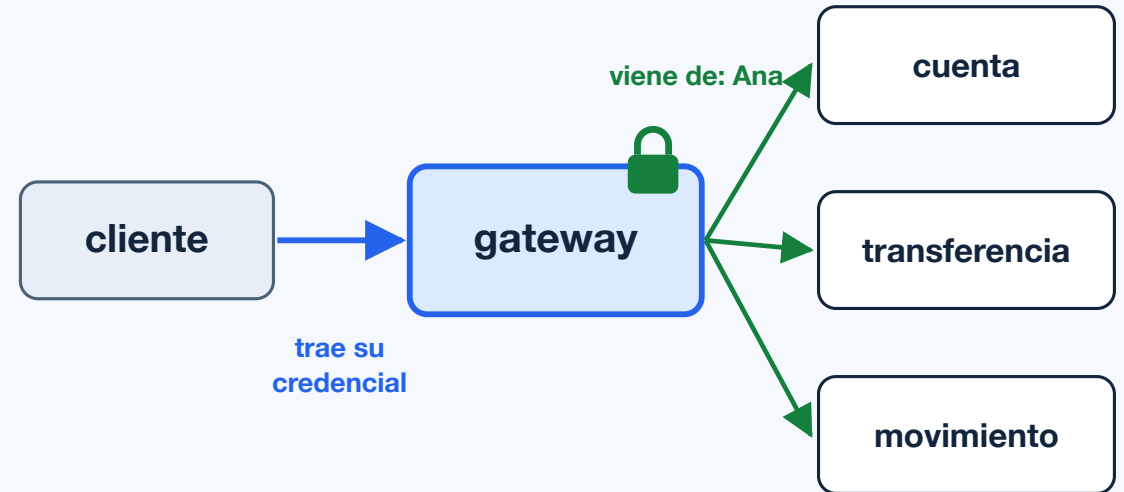
Autenticación: la puerta revisa quién eres, una sola vez

SIN PUERTA



Cada servicio revisa la credencial por su cuenta

CON PUERTA



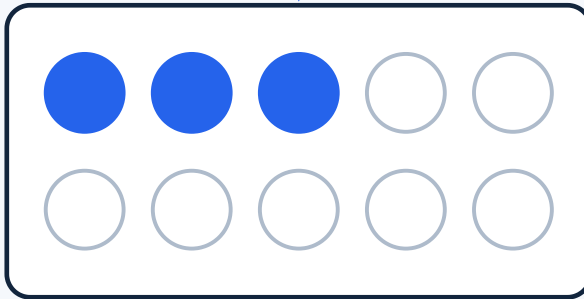
Se revisa una vez; los servicios reciben quién es

La puerta sabe quién eres. Si esa cuenta es tuya lo decide cuenta: eso ya es regla del negocio

Límite de peticiones: cada cliente trae una cubeta de fichas

La cubeta de la app de un tercero

↓ +5 fichas cada segundo



caben 10 · quedan 3

una cubeta por cliente

Llega una petición y hay ficha

gasta una ficha y pasa

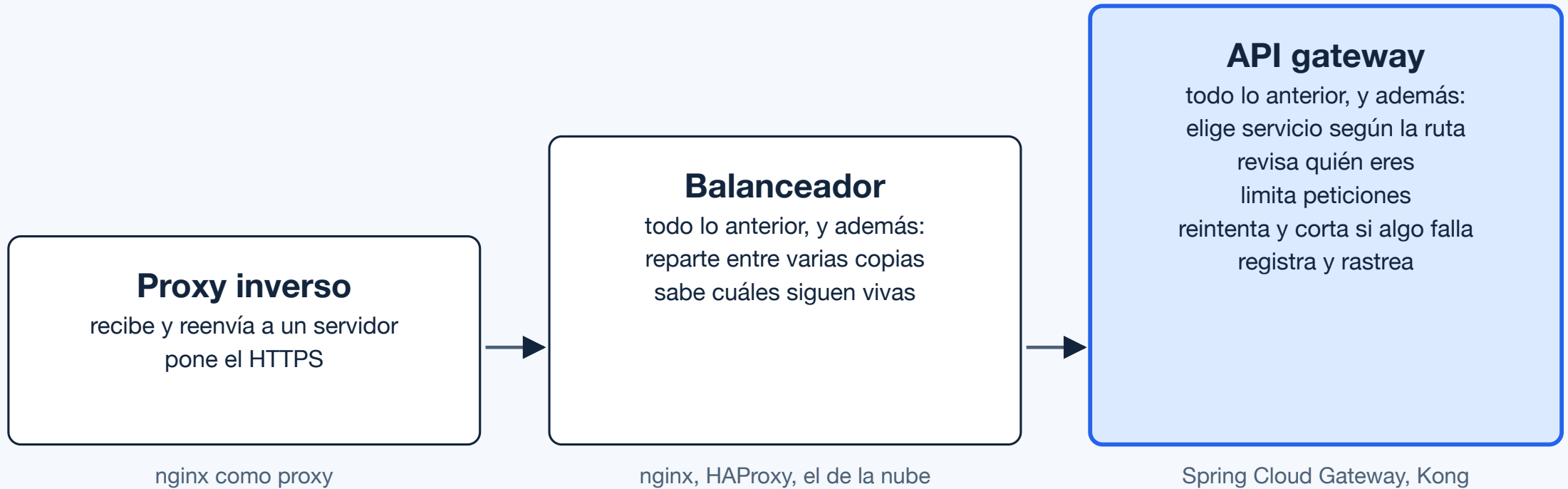
Llega una petición y la cubeta está vacía

429 · intenta más tarde

En inglés, *rate limiting*. En Spring Cloud Gateway es un filtro con dos números: cuántas fichas entran por segundo y cuántas caben. Protege a los servicios de un cliente que pide de más, a propósito o por un error en su código.

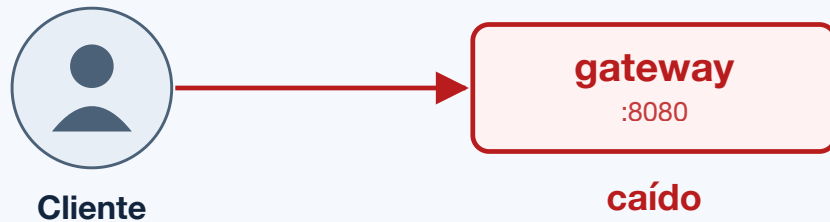
Sin esto, un solo cliente puede dejar sin servicio a todos los demás

Proxy, balanceador y gateway: cada uno hace lo del anterior y algo más



El balanceador elige copia; el gateway elige servicio. Se usa el más simple que alcance

Si se cae la puerta, se cae todo



transferencia :8083

cuenta :8081

movimiento :8082

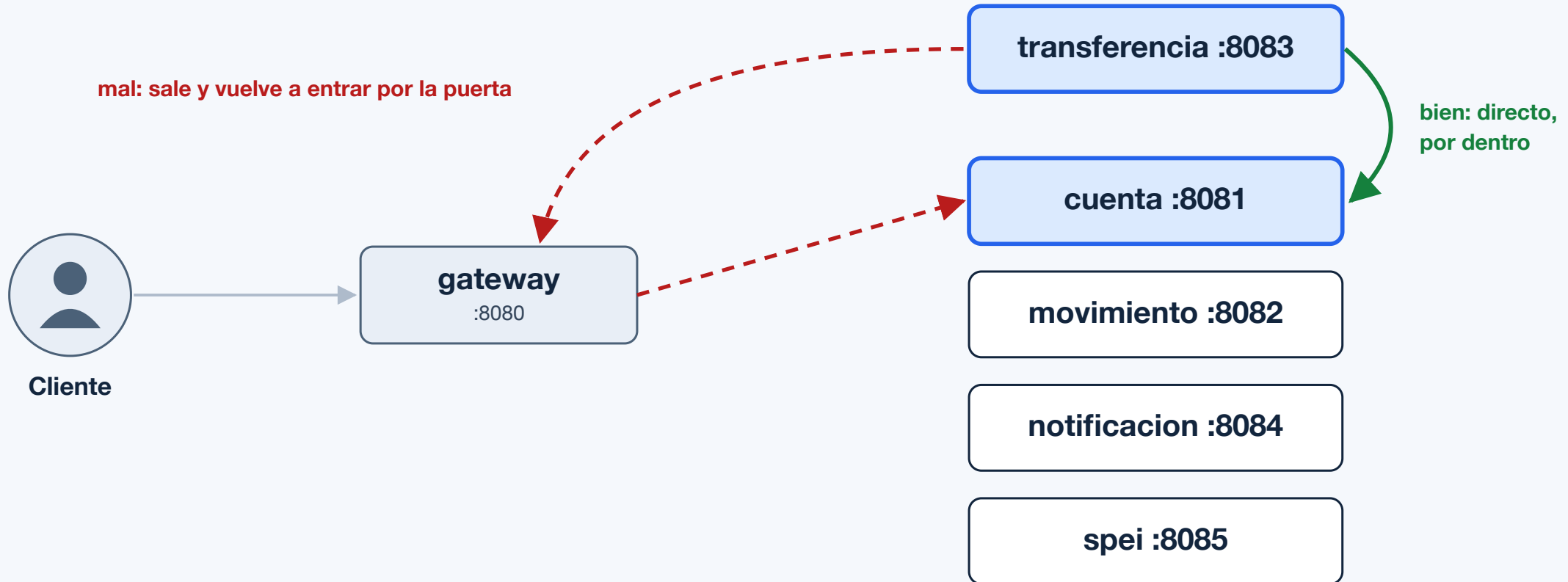
notificacion :8084

spei :8085

Se corren varias copias del gateway, y enfrente va un balanceador simple que reparte entre ellas. Se puede replicar porque no guarda estado.

Entre más cosas haga la puerta, más duele que se caiga

Entre servicios no se pasa por la puerta



La puerta es para lo que viene de fuera. Adentro, transferencia le habla directo a cuenta

Antes eran cinco puertos publicados; ahora, dos

```
$ docker compose ps
mexi-banco-v06-cuenta-1      8080/tcp
mexi-banco-v06-cuenta-2      8080/tcp
mexi-banco-v06-cuenta-3      8080/tcp
mexi-banco-v06-discovery-1    0.0.0.0:8761->8761/tcp
mexi-banco-v06-gateway-1     0.0.0.0:8080->8080/tcp
mexi-banco-v06-transferencia-1 8080/tcp

$ curl -s -o /dev/null -w "%{http_code}" localhost:8080/api/cuentas/000
404 ← lo contestó cuenta: la puerta supo llegar
```

Desde fuera solo se ven la puerta y el directorio. Los servicios quedaron dentro

Lo que ganas y lo que pagas

Ganas

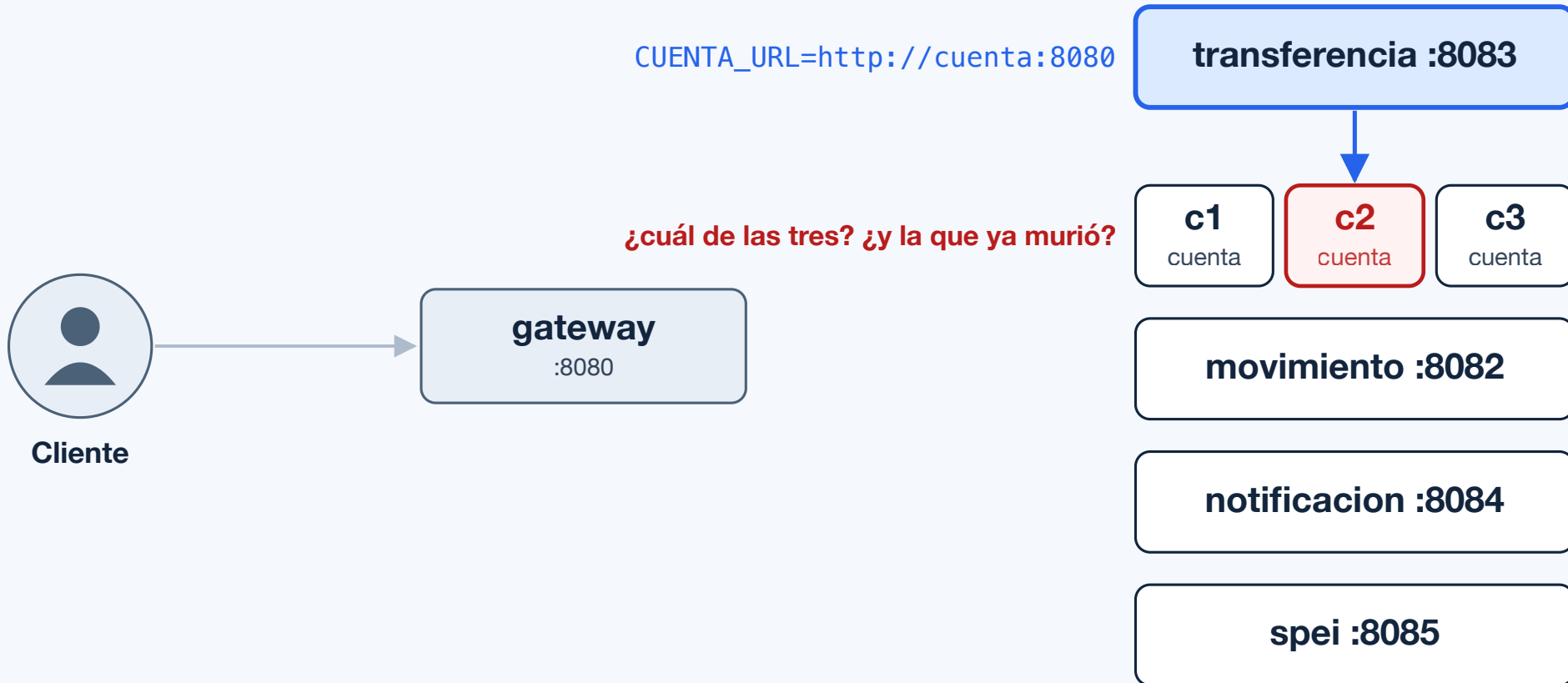
- Un solo puerto para el cliente
- Cambios internos que el cliente no ve
- Los controles comunes en un solo lugar

Pagas

- Un salto más de red en cada petición
- Un punto único de falla si no se replica
- Otra pieza que operar y vigilar

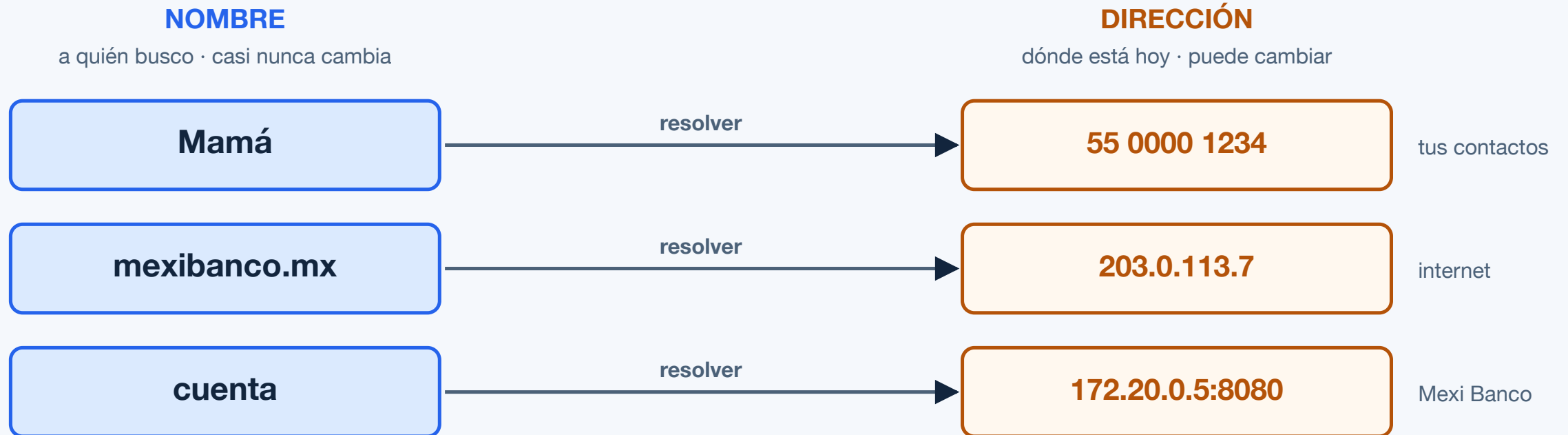
Pero la tabla dice "cuenta", y transferencia también la busca... ¿dónde está cuenta?

El nombre cuenta no dice cuántas copias hay ni cuáles siguen vivas



Un nombre fijo no basta cuando las copias entran y salen

Un nombre dice a quién buscas; una dirección dice dónde está



Resolver un nombre es convertirlo en dirección. ¿Quién hace esa conversión, y cuándo?

Enlazar: decidir a qué dirección apunta el nombre, y cuándo

Al programar

`"http://172.20.0.5:8080"`

la dirección va en el código

Si cambia: hay que recompilar

Al arrancar

`CUENTA_URL=http://...`

la dirección va en la configuración

Si cambia: hay que reiniciar

Al llamar

¿dónde está cuenta?

se pregunta en ese momento

Si cambia: se entera solo

enlace temprano

enlace tardío

En los libros: *naming* y *binding*. El enlace temprano también se llama estático; el tardío, dinámico.

Entre más tarde se enlaza, mejor aguanta los cambios. Discovery es enlazar al momento de llamar

El DNS es un directorio de nombres: preguntas, te contesta y guardas la respuesta



Ya es enlace tardío. Pero mientras la respuesta no venza, no vuelves a preguntar

Con copias que entran y salen, al DNS le faltan dos cosas

```
$ nslookup cuenta (con tres copias)
```

```
Address: 172.19.0.11
```

```
Address: 172.19.0.10 ← esta ya murió, y el DNS la sigue dando
```

```
Address: 172.19.0.5
```

```
quien pregunta guarda la respuesta un rato y no vuelve a preguntar
```

No sabe quién sigue vivo, y lo que guardaste puede estar viejo

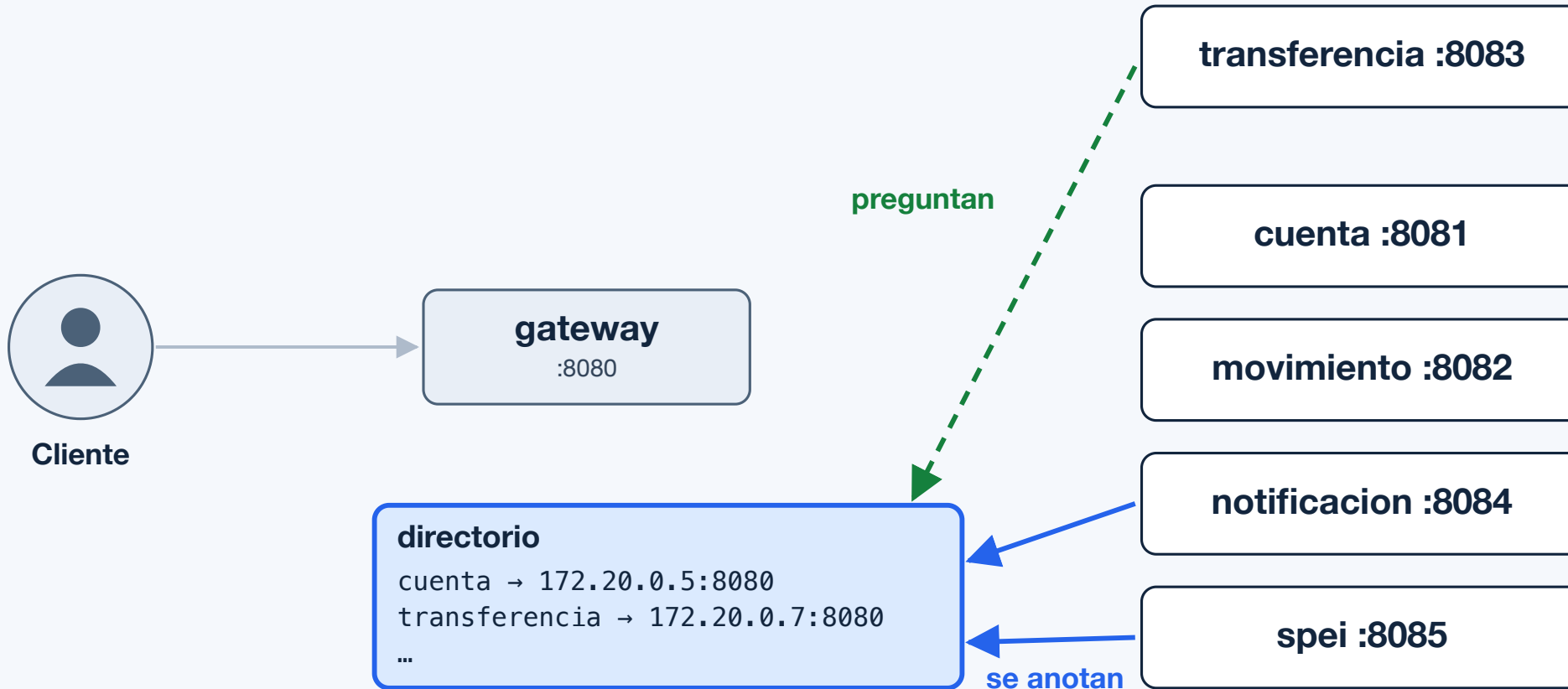
PREGUNTA

Se apaga una de las tres copias de cuenta. ¿Cuánto tarda el directorio en saberlo?

- A** Al instante
- B** Unos segundos
- C** Más de un minuto
- D** Nunca, hasta reiniciar

No es al instante: nadie le avisa. Se entera cuando deja de oírla, y eso tarda lo que dure el plazo

Un directorio donde cada servicio se anota y los demás preguntan



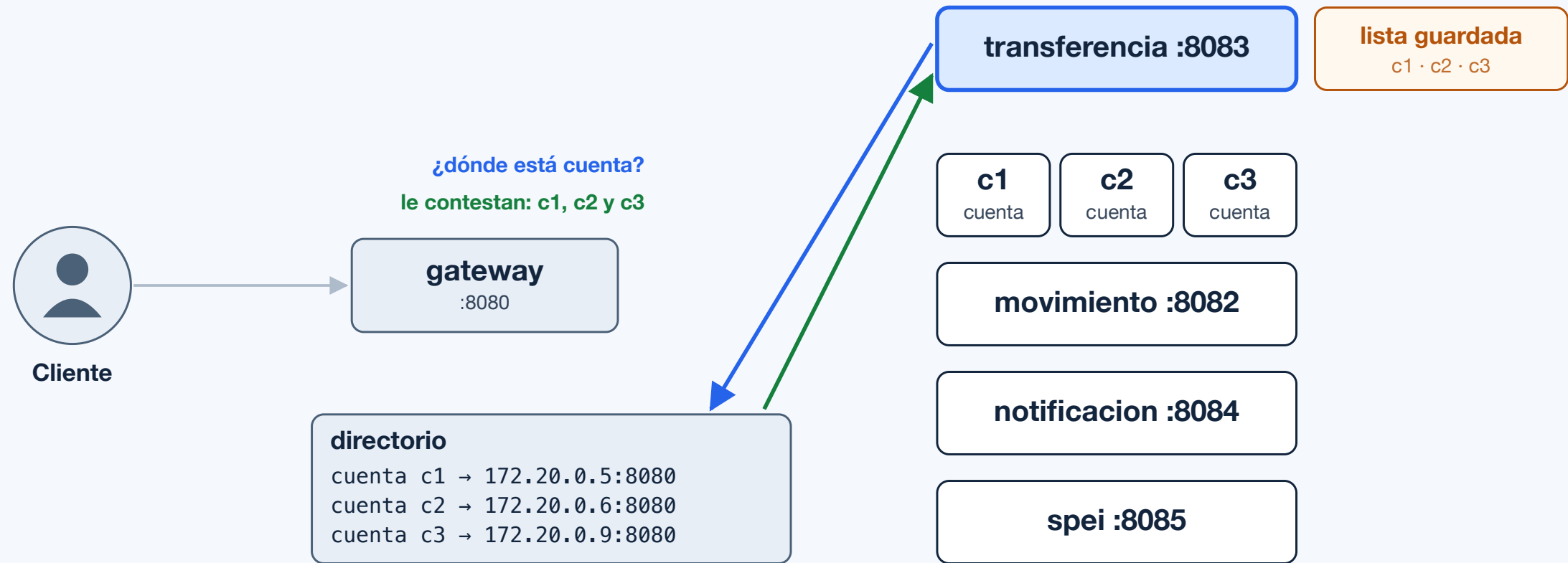
Es el mapa de sharding, ahora de servicios: nadie escribe direcciones, se pregunta por nombre

Cada copia de cuenta se anota sola y avisa que sigue viva



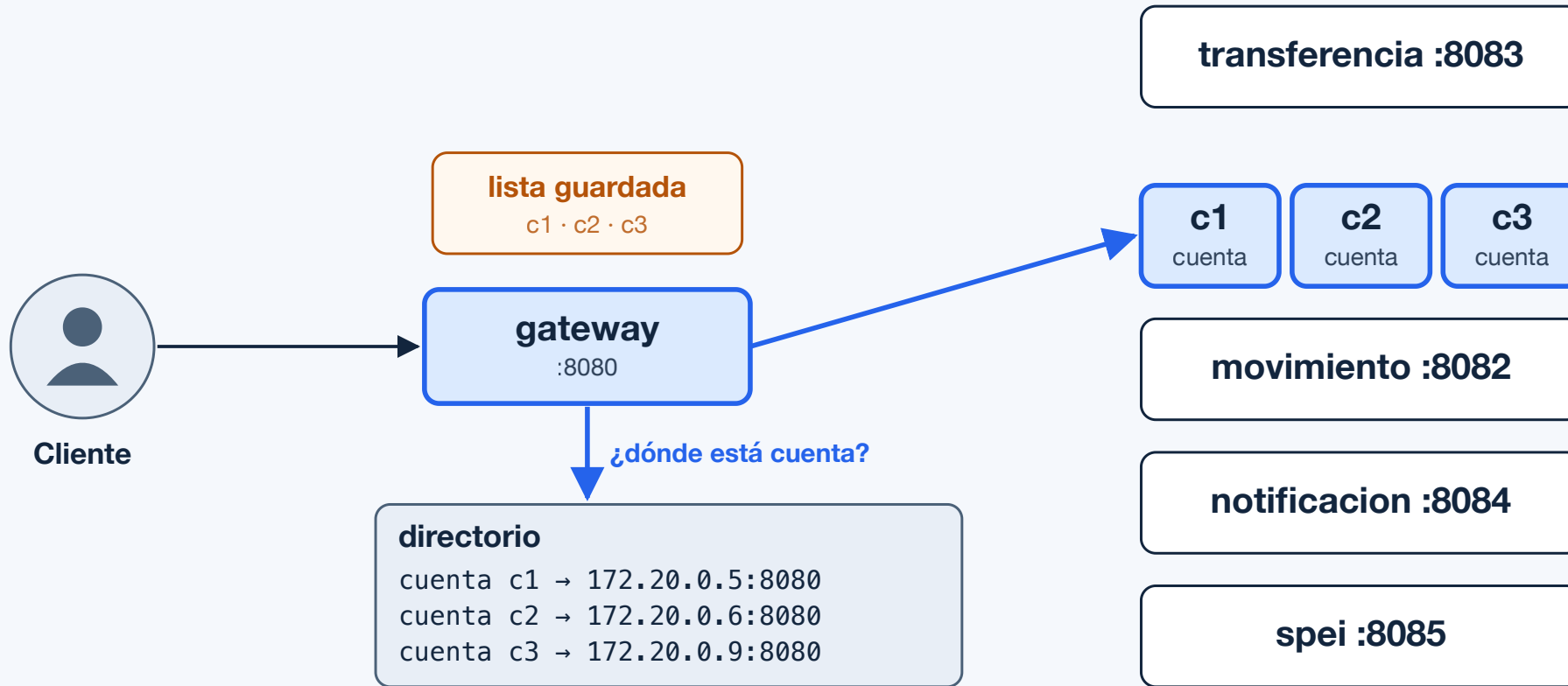
Un nombre, tres renglones. Ese aviso es un heartbeat, el mismo del líder que vimos en el repaso

Transferencia pregunta por cuenta, recibe la lista de las tres y la guarda



El directorio no elige: devuelve todas. Y transferencia no pregunta en cada llamada: usa su lista

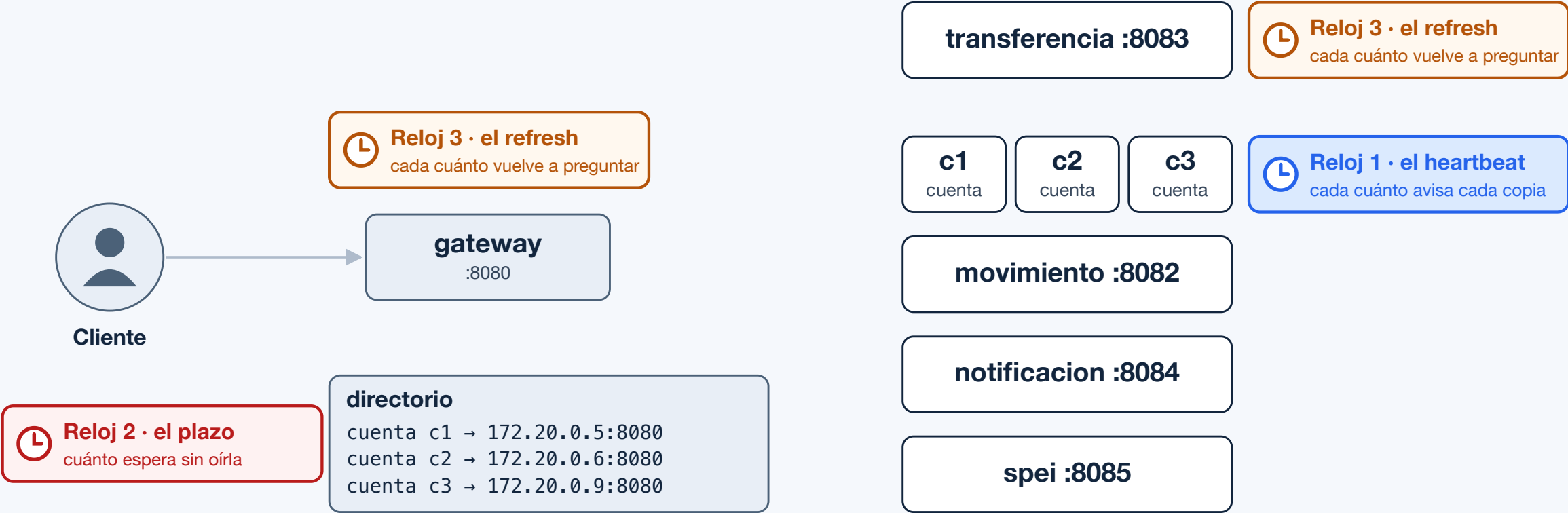
El gateway hace lo mismo: pregunta, recibe la lista y la guarda



Por eso la tabla de rutas del gateway trae nombres y no direcciones. Cada uno tiene su propia lista guardada: el gateway la suya y transferencia la suya.

Dos que preguntan: el gateway, para lo que viene de fuera; transferencia, para lo de dentro

Tres relojes distintos, en tres lugares distintos



El heartbeat lo marca cada copia; el plazo, el directorio; el refresh, cada uno de los que preguntan

Una copia se cae y deja de mandar su heartbeat



Si se apaga con orden, alcanza a avisar y sale del directorio al momento. Si muere de golpe, nadie avisa.

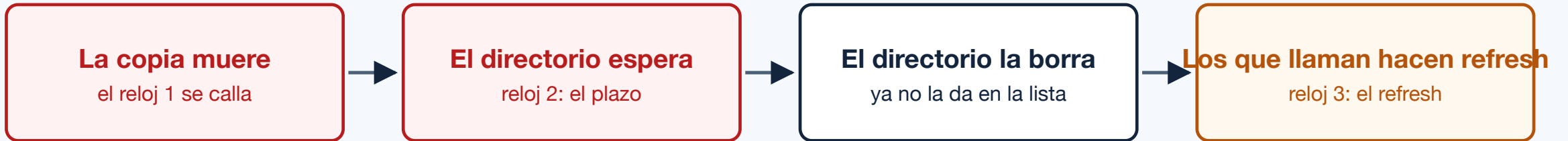
El directorio no ve la caída: solo deja de oírla. Y la sigue teniendo en sus tres renglones

Cada quien se entera a su tiempo

	directorio lo que tiene anotado	gateway su lista	transferencia su lista
c2 muere nadie lo sabe todavía	c1c2c3	c1c2c3	c1c2c3
Vence el plazo reloj 2: el directorio la borra	c1c3	c1c2c3	c1c2c3
El gateway hace refresh reloj 3: por la puerta ya no le llegan	c1c3	c1c3	c1c2c3
Transferencia hace refresh reloj 3: ya nadie le habla	c1c3	c1c3	c1c3

Durante un rato hay tres listas distintas. Mientras c2 siga en una, parte de las peticiones falla

Lo que tarda una copia muerta en dejar de recibir peticiones



lo que tarda en dejar de recibir peticiones = el plazo + el refresh

Ejemplo: con un plazo de 90 s y un refresh de 30 s, hasta 2 minutos

El heartbeat no suma: solo tiene que ser más corto que el plazo

Plazo corto: se entera rápido, pero da de baja a una copia que solo iba lenta. Plazo largo: al revés

Si lo escribiéramos nosotros: un diccionario y cuatro funciones

```
anotados = {}      # nombre -> {dirección: hora de su último heartbeat}

def anotar(nombre, direccion):      # lo llama el servicio al arrancar
    anotados[nombre][direccion] = ahora()

def heartbeat(nombre, direccion):  # lo llama cada cierto tiempo
    anotados[nombre][direccion] = ahora()

def donde_esta(nombre):            # lo preguntan los demás
    return list(anotados[nombre])

def barrer():                      # corre solo, cada tanto
    for copias in anotados.values():
        for direccion, visto in list(copias.items()):
            if ahora() - visto > PLAZO:
                del copias[direccion]
```

Lo difícil no está aquí: es que el directorio no se caiga y que nadie use una lista vieja

La misma idea, en tres tecnologías

Eureka

el del lab · con Spring Cloud

CÓMO SE ANOTA

El servicio trae una biblioteca y se registra solo al arrancar

CÓMO SE PREGUNTA

La biblioteca baja la lista y la guarda. En el gateway: lb://cuenta

CÓMO SABE QUE MURIÓ

Dejó de mandar heartbeat y venció el plazo

Consul

un agente junto a cada servicio

CÓMO SE ANOTA

El agente lo registra; el servicio no cambia

CÓMO SE PREGUNTA

Por HTTP o como si fuera DNS

CÓMO SABE QUE MURIÓ

El agente revisa su salud cada tanto

Kubernetes

viene con la plataforma

CÓMO SE ANOTA

La plataforma lo anota: no se toca el código

CÓMO SE PREGUNTA

Por DNS: http://cuenta

CÓMO SABE QUE MURIÓ

La plataforma revisa su salud y lo saca de la lista

En las tres: anotarse, preguntar y darse de baja. Cambia quién hace el trabajo

Con Eureka y sus tiempos de fábrica: casi cuatro minutos

197 s

hasta que Eureka la borró
(configurado: 90 s; su código
lo duplica)

+

51 s

de listas guardadas en el
gateway y en transferencia

=

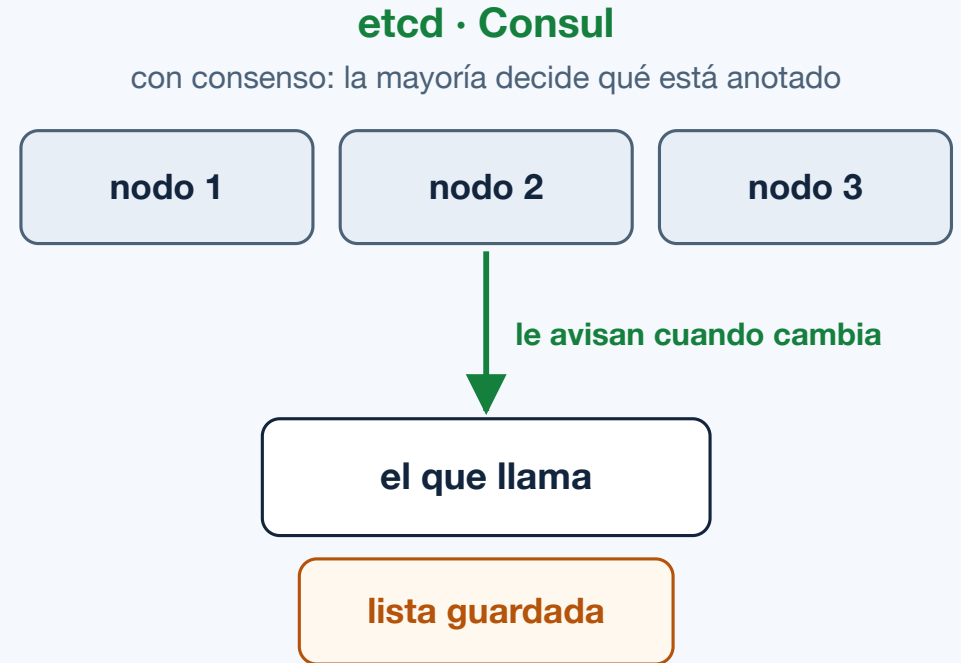
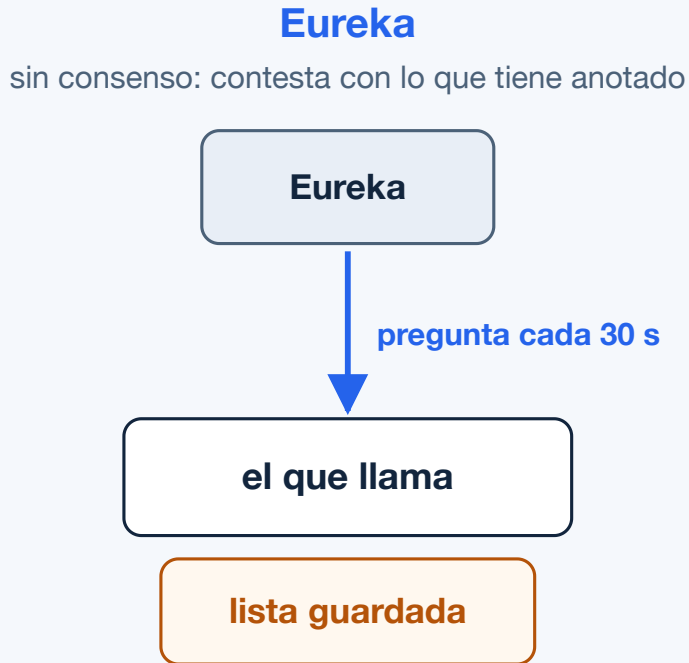
≈ 4 min

mandándole peticiones a
una copia muerta

Medido en Mexi Banco, etapa 03: heartbeat cada 30 s, plazo de 90 s y refresh de la lista cada 30 s, matando una copia de cuenta con docker kill

Respuesta: C. En el lab bajamos los tiempos y sale en ≈ 35 s

Con o sin consenso, el que llama lee de una copia



En Kubernetes el directorio vive en etcd, pero los servicios no le preguntan a etcd: preguntan por DNS, y la plataforma mantiene esa lista al día.

Para un directorio, contestar rápido con algo un poco viejo es mejor que no contestar

Seis nombres anotados, y cuenta con tres copias

Application	Status
CUENTA	UP (3)
GATEWAY	UP (1)
MOVIMIENTO	UP (1)
NOTIFICACION	UP (1)
SPEI	UP (1)
TRANSFERENCIA	UP (1)

El tablero del directorio en la etapa 03, en `http://localhost:8761`

Nadie escribió esas direcciones: cada copia se anotó sola al arrancar

Lo que ganas y lo que pagas

Ganas

Nadie escribe direcciones

Las copias entran y salen solas

El gateway también pregunta por nombre

Pagas

Otra pieza que puede caerse

La información llega tarde: plazo más lista guardada

Una dirección muerta de vez en cuando

Ya sabemos dónde están las tres copias. ¿A cuál le hablo?

Tres copias de cuenta y una sola petición



Ya tengo la lista de las tres. Si todas las peticiones van a c1, las otras dos no sirven de nada

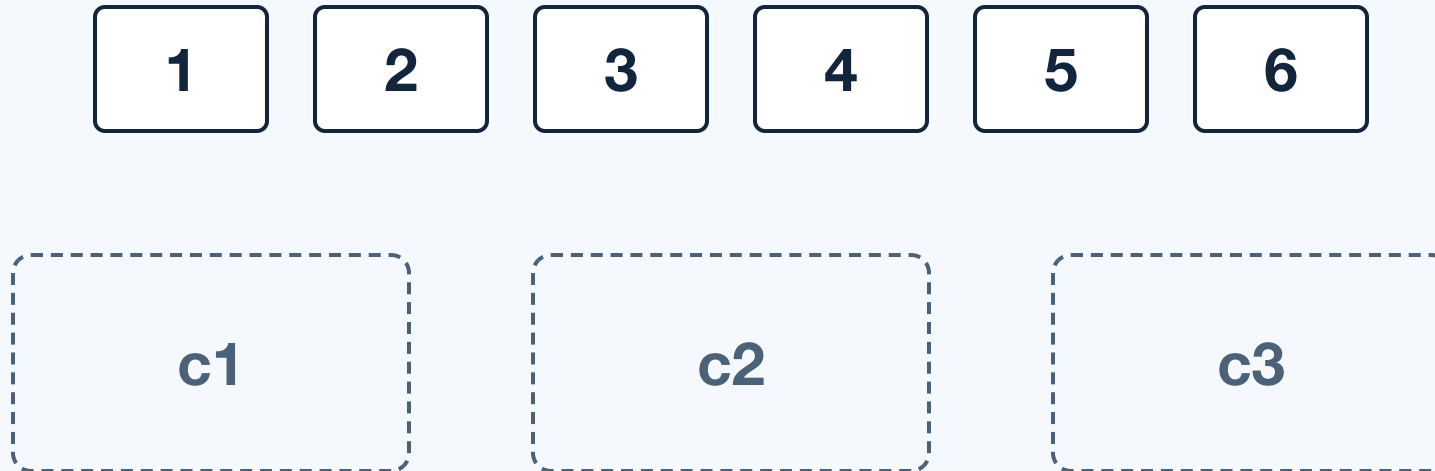
Se puede repartir porque cuenta no recuerda nada entre peticiones



Sin estado, cualquiera sirve. Con estado es el problema del shard: solo una tiene el dato

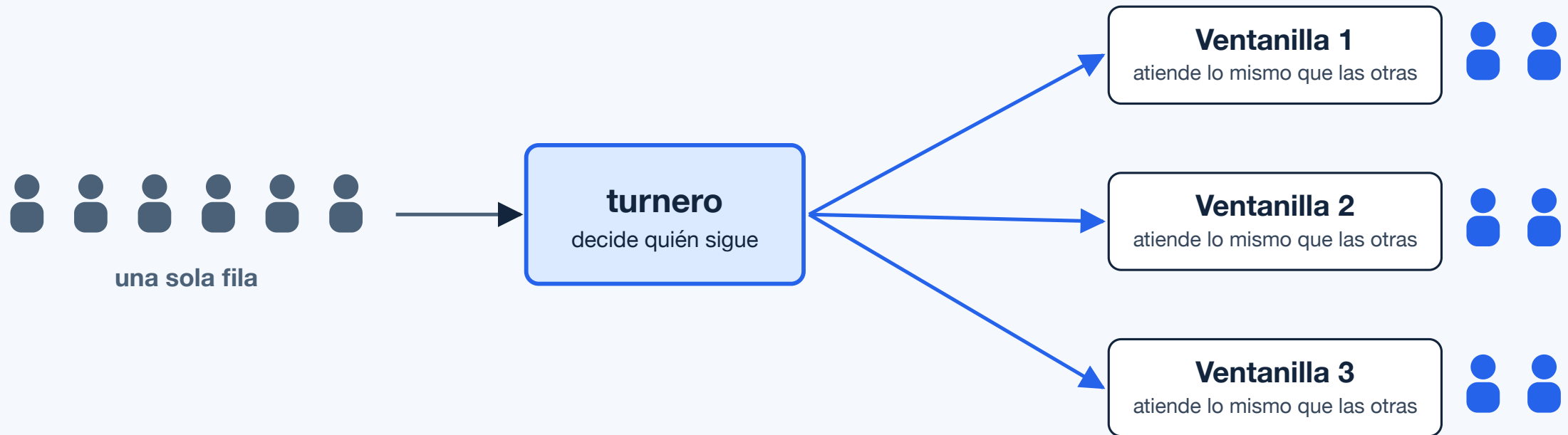
PREGUNTA

Llegan 6 peticiones y las 3 copias están sanas. ¿Cómo las repartes?



Una línea en el chat: su regla

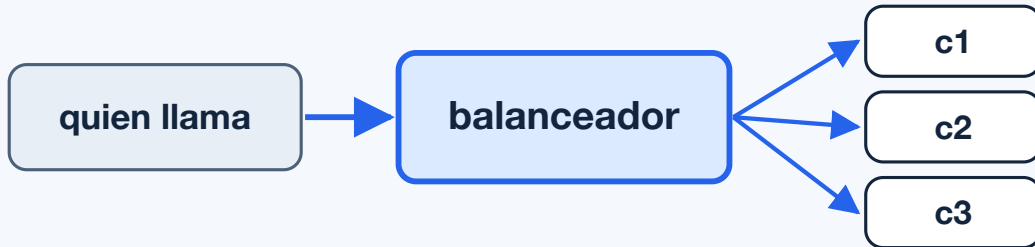
Una sola fila y un turnero que reparte entre ventanillas iguales



En sharding las ventanillas no eran iguales: cada una tenía lo suyo. Aquí sí, y por eso se reparte

Decide alguien enfrente, o decide el que llama

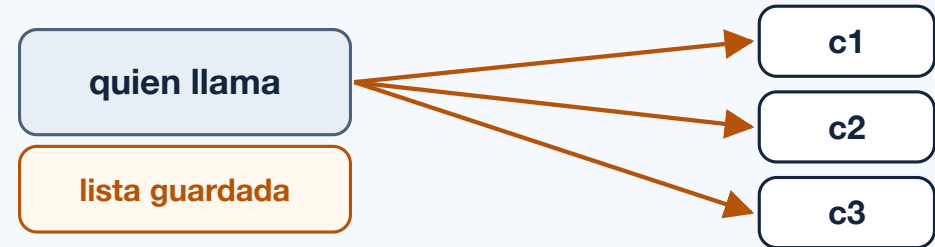
ALGUIEN ENFRENTA del lado del servidor



Gana: una sola dirección para todos los que llaman

Paga: un salto más; y si se cae, se cae todo

EL QUE LLAMA del lado del cliente



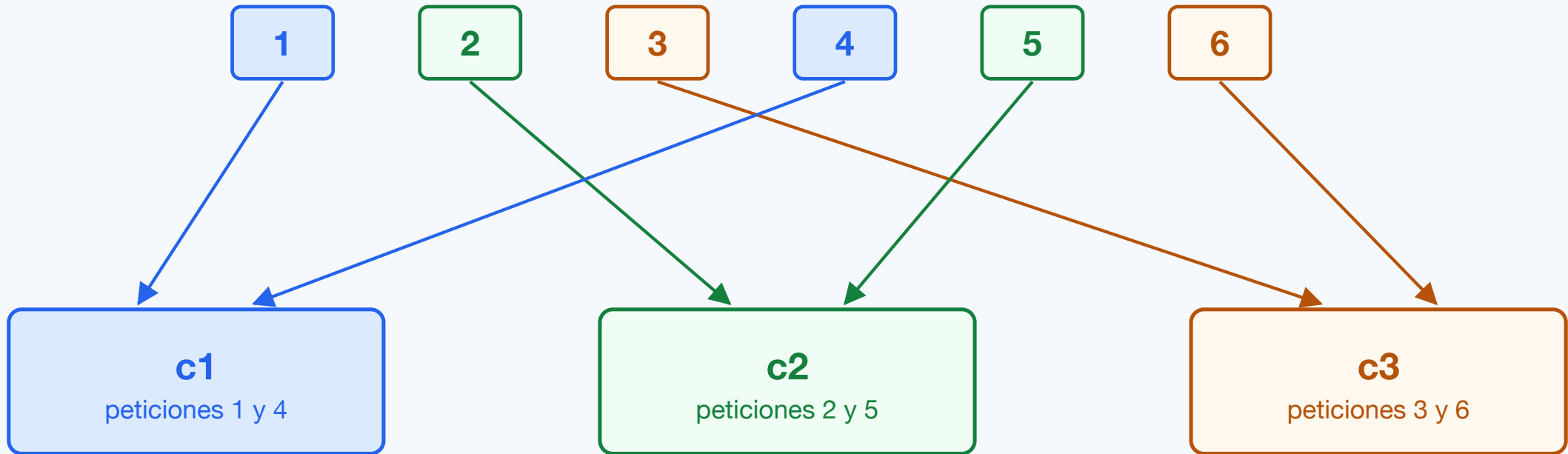
Gana: sin salto de más, y nada enfrente que se caiga

Paga: cada quien lo implementa; su lista puede estar vieja

Lo habitual es poner a alguien enfrente: lo aprovechan todos los que llaman sin que cada uno lo programe.

En Mexi Banco hay los dos: el gateway reparte para la app y transferencia reparte para sí misma

Por turnos: uno para cada copia y vuelve a empezar



Es la regla más simple: no hay que medir nada. Cada copia recibe el mismo número de peticiones

Round robin reparte parejo las peticiones, no el trabajo

Las mismas seis peticiones, por turnos. A la derecha, cuánto tarda en atenderse cada una.

c1

1 · estado de cuenta anual 8 s

4 · estado de cuenta anual 8 s

2 peticiones
16 s de trabajo

c2

2 · consultar saldo 1 s

5 · consultar saldo 1 s

2 peticiones
2 s de trabajo

c3

3 · consultar saldo 1 s

6 · consultar saldo 1 s

2 peticiones
2 s de trabajo

Es lo mismo que vimos con el hash: parejo en número, disparejo en trabajo

Cada regla ataca un problema distinto

Si una máquina es más grande que las otras

Ponderado

más turnos a la grande

SU LÍMITE

los pesos son fijos y la carga cambia

Si unas peticiones duran mucho más que otras

Menos conexiones

a la que tiene menos abiertas

SU LÍMITE

una conexión puede costar más que otra

Si hay muchos repartidores que no se hablan

Al azar

cualquiera, sin llevar la cuenta

SU LÍMITE

con pocas peticiones sale disparejo

Si todos corren a la misma menos ocupada

Dos al azar

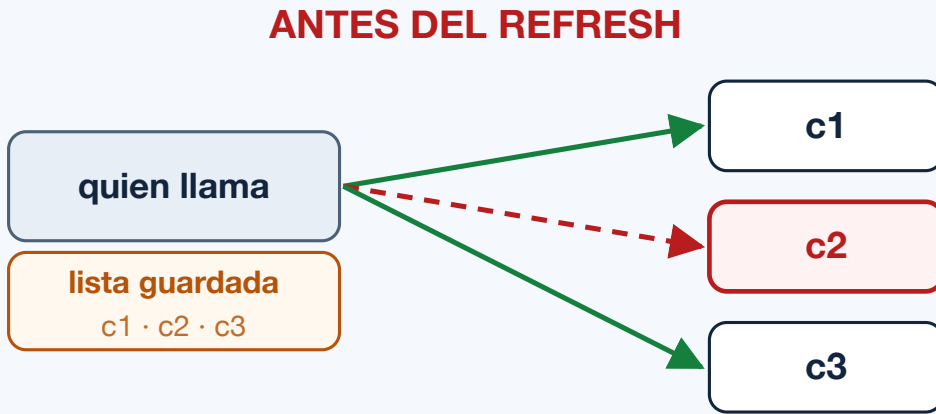
tomas dos y eliges la menos ocupada

SU LÍMITE

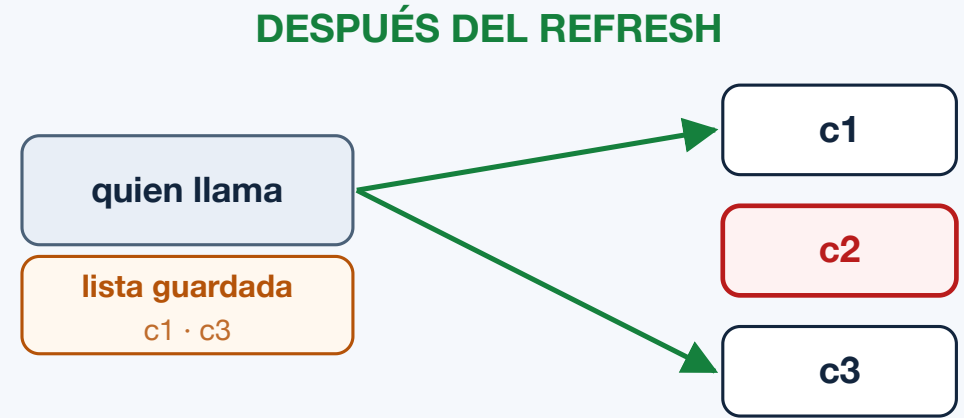
necesita alguna medida de carga

Ninguna ve el trabajo real de cada copia. Y el que llama solo ve las peticiones que él mismo mandó

No se reparte a quien ya salió de la lista



Una de cada tres peticiones va a la copia muerta



Los turnos quedan: c1, c3, c1, c3...

Un balanceador enfrente puede preguntarle él mismo a cada copia si está sana (*health check*). El que llama depende de lo que le diga el directorio.

¿Quién avisa que c2 murió? El directorio, con el retraso que ya vimos: plazo más refresh

Si lo escribiéramos nosotros: una lista y un contador

```
copias = ["c1", "c2", "c3"]          # la lista que dio el directorio
turno = 0

def elegir_por_turnos():              # round robin
    global turno
    copia = copias[turno % len(copias)]
    turno += 1
    return copia

def elegir_menos_ocupada():           # menos conexiones
    return min(copias, key=lambda c: abiertas[c])

def llamar(peticion):
    return enviar(elegir_por_turnos(), peticion)
```

Cambiar de regla es cambiar una función. Lo difícil es que la lista esté al día

El mismo reparto, en tres tecnologías

Spring Cloud LoadBalancer

el del lab · reparte el que llama

```
@Bean
@LoadBalanced
RestClient.Builder
    restClientBalanceado(...) { ... }

// y se llama por nombre:
"http://cuenta"
```

Usa la lista del directorio; round robin

nginx

reparte alguien enfrente

```
upstream cuenta {
    least_conn;
    server c1:8080;
    server c2:8080;
    server c3:8080;
}
```

Sin least_conn, reparte por turnos

Kubernetes

un Service · reparte la plataforma

```
kind: Service
metadata:
  name: cuenta
spec:
  selector:
    app: cuenta
  ports:
    - port: 8080
```

Reparte entre las copias sanas

En las tres se dice a quiénes repartir y con qué regla. Se configura, no se programa

En la nube, el balanceador también se renta ya hecho

AWS

de aplicación: lee la petición y su ruta

Application Load Balancer

de red: solo ve dirección y puerto

Network Load Balancer

Google Cloud

de aplicación: lee la petición y su ruta

Application Load Balancer

de red: solo ve dirección y puerto

Network Load Balancer

Azure

de aplicación: lee la petición y su ruta

Application Gateway

de red: solo ve dirección y puerto

Load Balancer

Lo que cambia

No corres el balanceador: el proveedor lo mantiene y lo replica

Él mismo revisa la salud de cada copia

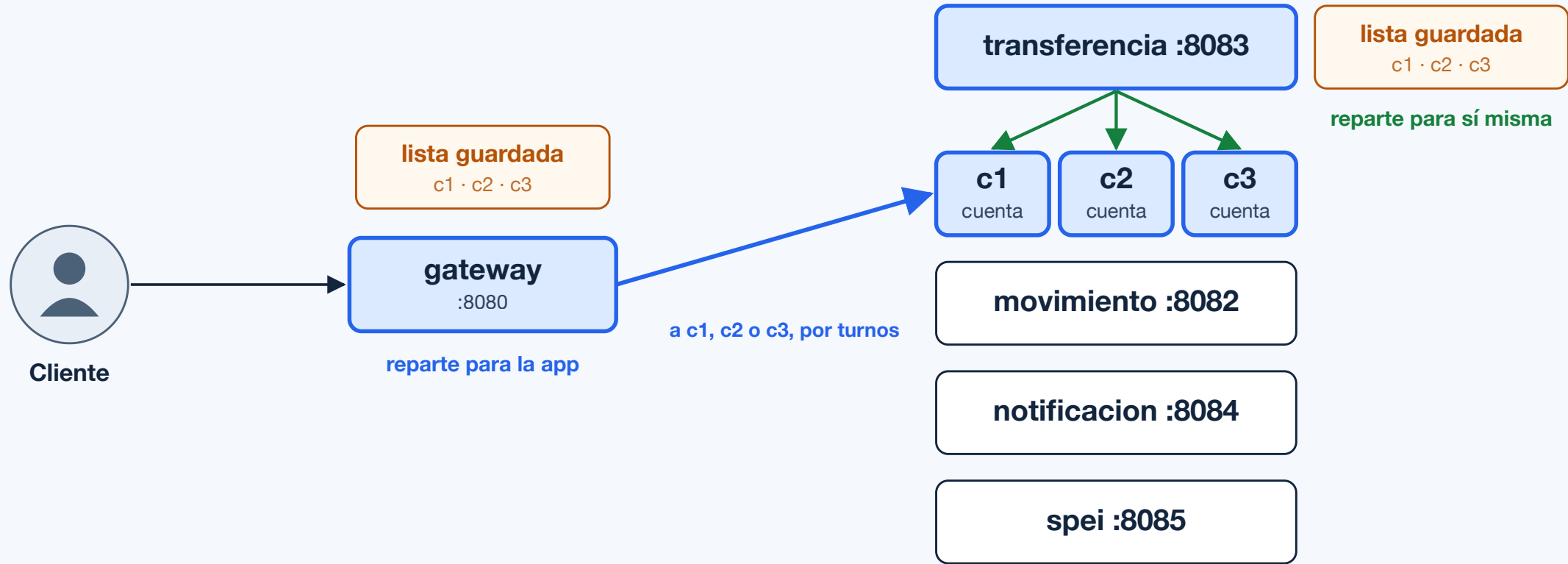
Lo que pagas

Se cobra por hora y por tráfico

Quedas atado a ese proveedor

El de red es más simple y rápido; el de aplicación puede mirar la ruta, como un gateway

En la etapa 03 hay dos repartos, cada uno con su turno



En el lab van a ver alternar la cabecera **X-Instancia** en seis llamadas al gateway: ese es el turno del gateway. El turno puede empezar en cualquier copia.

El gateway reparte como alguien enfrente; transferencia reparte como el que llama

Seis llamadas, tres copias, por turnos

```
$ for i in 1 2 3 4 5 6; do curl -s -D - -o /dev/null \
    localhost:8080/api/cuentas/000 | grep -i x-instancia; done
```

X-Instancia: 3486d8704bc4

X-Instancia: 0e8f507e4775

X-Instancia: 15896c4dbde3

X-Instancia: 3486d8704bc4

X-Instancia: 0e8f507e4775

X-Instancia: 15896c4dbde3

Cada respuesta dice qué copia contestó. Ese es el turno del gateway

Lo que ganas y lo que pagas

Ganas

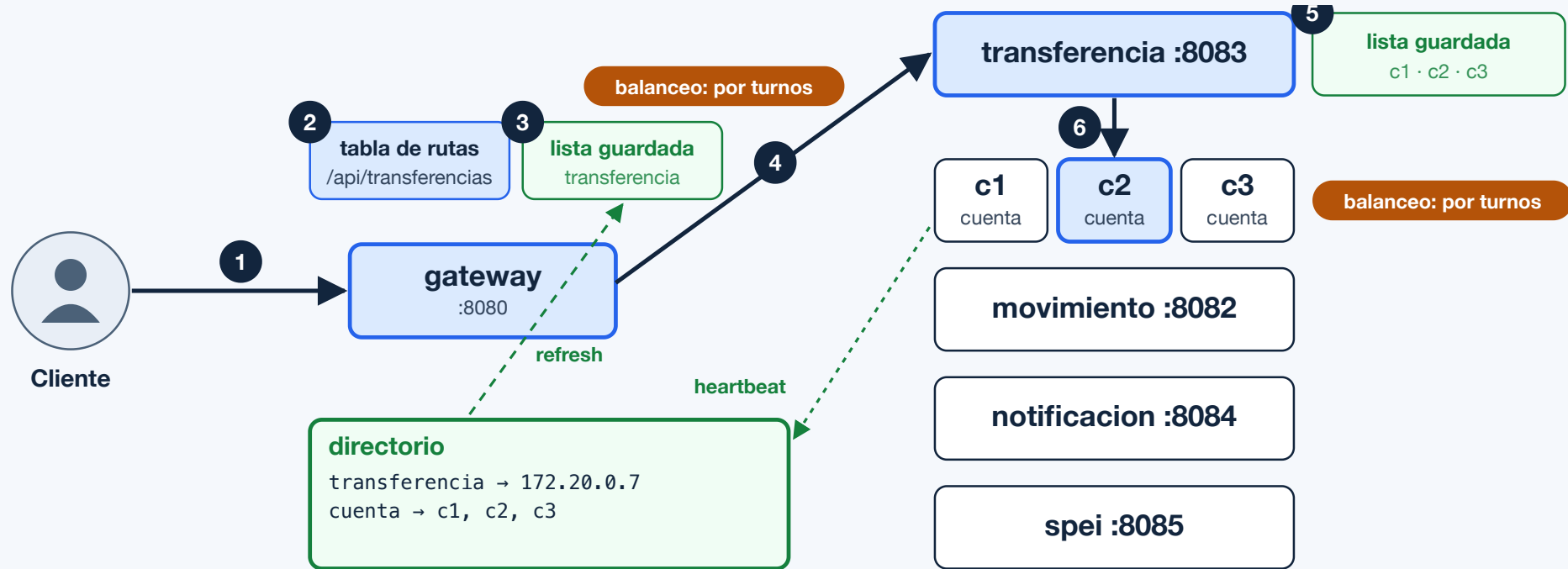
Más capacidad: tres copias atienden más
Una copia cae y las otras siguen

Pagas

El reparto depende de una lista que puede estar vieja
El estado tiene que vivir fuera de las copias
La regla no ve el trabajo real de cada copia

Tres piezas, y cada una es algo más que puede fallar. Veámoslas trabajar juntas

El viaje de una transferencia, con las tres piezas



- 1 · La app manda `POST /api/transferencias` al gateway
- 3 · **Directorio**: la lista guardada dice dónde están sus copias
- 5 · **Directorio**: transferencia busca cuenta en su lista guardada

- 2 · **Gateway**: su tabla de rutas dice que va a transferencia
- 4 · **Balanceo**: elige una copia por turnos y reenvía
- 6 · **Balanceo**: elige c2 por turnos y le pide el cargo

El balanceo no es una caja aparte: vive dentro de quien llama, en el gateway y en transferencia

En cada salto se contestan las mismas tres preguntas



Por eso son tres piezas: cada una contesta una pregunta. Al volver del descanso, ustedes las arman

30 MINUTOS

Descanso

30

Al volver: el laboratorio, con su guía aparte

Cada equipo deja su ficha antes de salir

Campo	Qué escriben
Equipo	Los cuatro nombres
Caso	Qué sistema y para quién, en dos líneas
Módulos	Las partes del monolito (como cuenta, transferencia...)
Endpoints	Los principales, con su verbo
Monolito inicial	Lenguaje, framework y base de datos

¿Dónde la suben?

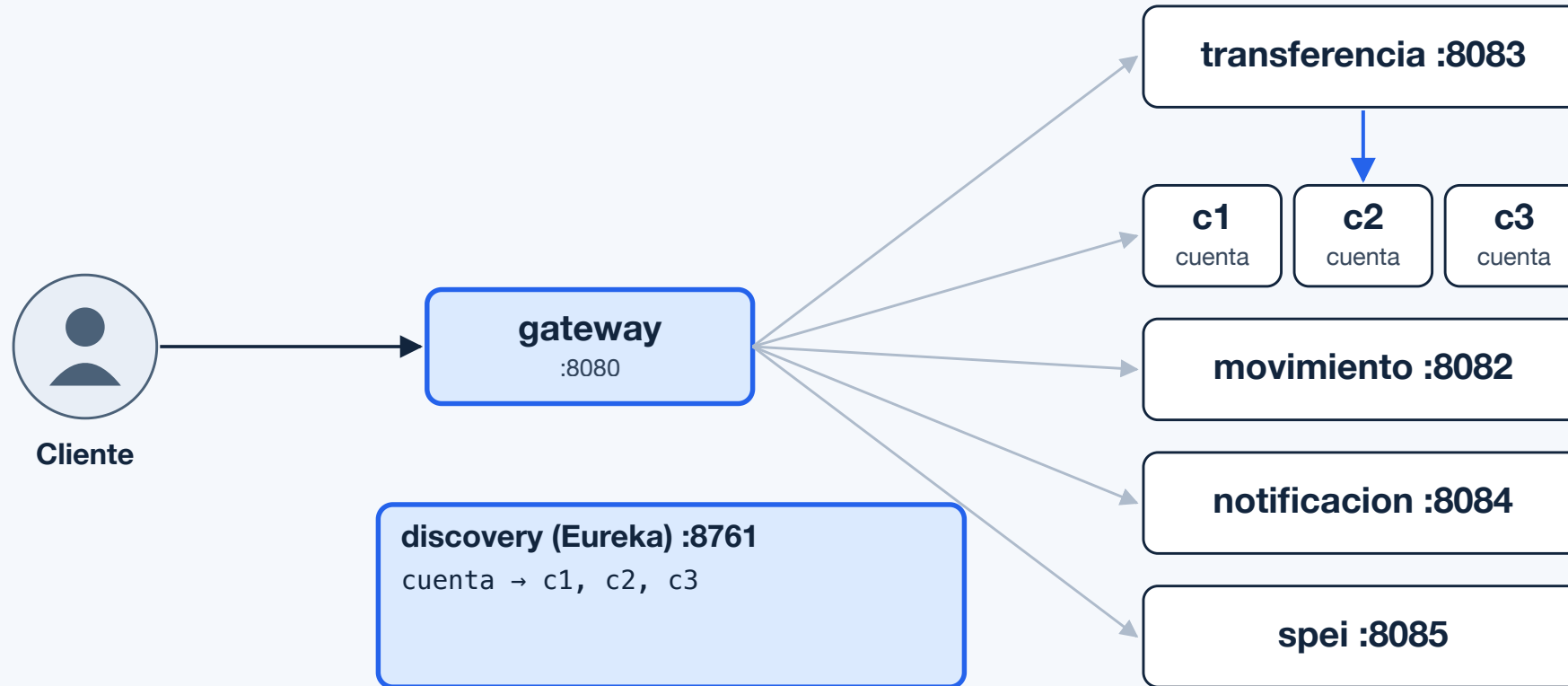
Diez minutos, en equipo; la reviso esta semana

La lectura de la semana

Fowler, CircuitBreaker, y Dapper (2010), secciones 1 a 3
2 preguntas y 1 hallazgo en Discussions · en tiempo hasta el sábado 10 de octubre, 06:59

La S08: qué hacer cuando estas piezas fallan

Una puerta, un directorio y alguien que reparte; cada uno es algo más que puede caerse



La próxima clase: qué hacer cuando esas piezas fallan